



Numerical approximation of partial differential equations with MFEM library

Felipe Cruz Vasquez

Universidad Nacional de Colombia
Faculty of Sciences
Department of Mathematics
Undergraduate Program in Mathematics
Bogotá D.C., Colombia
2021

Numerical approximation of partial differential equations with MFEM library

Felipe Cruz Vasquez

Undergraduate degree work submitted as a partial requirement for the degree
of:

Mathematician

Advisor:

Dr. Boyan Lazarov
Research Engineer - Computational Engineering Division
Lawrence Livermore National Laboratory

Co-advisor:

Prof. Dr. Juan Carlos Galvis Arrieta
Full Professor - Department of Mathematics
Faculty of Sciences
Universidad Nacional de Colombia

Research line:

Finite Element Methods

Universidad Nacional de Colombia
Faculty of Sciences
Department of Mathematics
Undergraduate Program in Mathematics
Bogotá D.C., Colombia
November 29, 2021

ABSTRACT. We revise the finite element formulation for Lagrange, Raviart-Thomas, and Taylor-Hood finite element spaces. We solve Laplace equation in first and second order formulation, and compare the solutions obtained with Lagrange and Raviart-Thomas finite element spaces by changing the order of the shape functions and the refinement level of the mesh. Finally, we solve Navier-Stokes equations in a two dimensional domain, where the solution is a steady state, and in a three dimensional domain, where the system presents a turbulent behaviour. All numerical experiments are computed using MFEM library, which is also studied.

Keywords: Finite Elements, MFEM library, Lagrange, Raviart-Thomas, Taylor-Hood, Laplace Equation, Navier-Stokes Equations..

Título en español: Aproximación numérica de ecuaciones diferenciales parciales con la librería MFEM

RESUMEN. Revisamos la formulación de elementos finitos para los espacios de elementos finitos de Lagrange, Raviart-Thomas y Taylor-Hood. Solucionamos la ecuación de Laplace en su formulación de primer y segundo orden, y comparamos las soluciones obtenidas con los espacios de elementos finitos de Lagrange y Raviart-Thomas al cambiar el orden de las funciones base y el nivel de refinamiento de la malla. Finalmente, resolvemos las ecuaciones de Navier-Stokes en un dominio bidimensional, donde la solución es un estado estable, y en un dominio tridimensional, donde el sistema presenta un comportamiento turbulento. Todos los experimentos numéricos se realizan utilizando la librería MFEM, la cual es también estudiada.

Palabras claves: Elementos Finitos, Librería MFEM, Lagrange, Raviart-Thomas, Taylor-Hood, Ecuación de Laplace, Ecuaciones de Navier-Stokes..

Contents

1 Preliminaries	5
1.1 Partial Differential Equations	5
1.1.1 Laplace equation	5
1.1.2 Navier-Stokes equations	7
1.2 The Finite Element Method	8
1.2.1 FEM for elliptic problems	9
1.2.2 Lagrange finite elements	13
1.2.3 Lagrange spaces of higher order	18
1.2.4 Raviart-Thomas finite elements	19
1.2.5 Taylor-Hood finite elements	23
1.3 MFEM Library	26
1.3.1 Information about the library	26
1.3.2 Overview	27
1.3.3 Code structure	28
2 Lagrange vs. Raviart-Thomas finite elements	29
2.1 Problem	29
2.2 Code	30
2.3 Tests	30
2.4 Results	32
2.5 Analysis	35
2.6 Some other examples	36
3 Numerical Experiments with NS	40
3.1 2D Experiment: Steady State	40
3.2 3D Experiment: Turbulence	44
4 Conclusion and Perspectives	47
5 Appendices	49
5.1 Appendix A : Code for comparison	49
5.2 Appendix B : Numerical values of the comparison	59
5.3 Appendix C : MiniApp Code for Navier-Stokes	61

1. Preliminaries

In this section we are going to recall the theoretical background needed in the rest of the paper. First, we are going to review the finite element methods used for the Laplace equation in second and first order form. We write the strong and weak form of the problem and introduce the Lagrange and mixed finite element spaces. We also introduce the MFEM library by giving an overview of its main characteristics and the general structure of a finite element code in MFEM.

1.1. Partial Differential Equations

For the scope of this work, partial differential equations are of the form

$$F\left(u, p, \frac{\partial p}{\partial x_i}, \frac{\partial u}{\partial x_i}, \frac{\partial^2 p}{\partial x_i^2}, \frac{\partial^2 u}{\partial x_i^2}, f\right) = 0; \quad i = 1, 2, 3, 4$$

where $u : \Omega \subseteq \mathbb{R}^3 \rightarrow U \subseteq \mathbb{R}^3$, $p : \Omega \subseteq \mathbb{R}^3 \rightarrow P \subseteq \mathbb{R}$, f is a restriction parameter and F is **any** mathematical expression within its variables.

It is well known that PDEs have multiple solutions, in fact, there is a vectorial space consisting of all the solutions for a given equation. For this reason, the equation is usually presented with a boundary condition that forces a unique solution for the equation [2]. If some problem is being solved in a given domain $\Omega \subseteq \mathbb{R}^3$, whose boundary is Γ , then the problem has the form

$$\begin{cases} F\left(u, p, \frac{\partial p}{\partial x_i}, \frac{\partial u}{\partial x_i}, \frac{\partial^2 p}{\partial x_i^2}, \frac{\partial^2 u}{\partial x_i^2}, f\right) = 0 & \text{in } \Omega, \\ F_0\left(u, p, \frac{\partial p}{\partial x_i}, \frac{\partial u}{\partial x_i}, \frac{\partial^2 p}{\partial x_i^2}, \frac{\partial^2 u}{\partial x_i^2}, g\right) = 0 & \text{in } \Gamma. \end{cases}$$

The two main equations that we treat are the Laplace equation and the Navier-Stokes equation, whose represent a fluids phenomenon in real life. Note that $x_1 = x, x_2 = y, x_3 = z, x_4 = t$ (in time-space interpretation) and that, p is the fluid's pressure and u is the fluid's velocity.

1.1.1. Laplace equation

Laplace equation consists on finding $p : \Omega \subseteq \mathbb{R}^3 \rightarrow \mathbb{R}$ such that

$$-\Delta p = f, \quad \text{in } \Omega \tag{1}$$

where $f : \Omega \rightarrow \mathbb{R}$ is a given function and $\Delta p = \nabla \cdot \nabla p = \frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} + \frac{\partial^2 p}{\partial z^2}$ [4]. It is clear that the equation is a partial differential equation of the form presented before in Section 1.1 because it only depends on the second order derivatives of p .

Now, we present two common ways of imposing a boundary condition to the problem: Dirichlet and Neumann. Let Γ be the boundary of Ω .

Dirichlet boundary condition

The Dirichlet boundary condition is

$$p = g \text{ in } \Gamma \quad (2)$$

where $g : \Gamma \rightarrow \mathbb{R}$ is a given function [4]. If $g = 0$, the condition is called homogeneous Dirichlet boundary condition.

Neumann boundary conditions

The Neumann boundary condition is

$$-\nabla p \cdot \eta = h \text{ in } \Gamma \quad (3)$$

where $h : \Gamma \rightarrow \mathbb{R}$ is a given function and η is the boundary's normal vector [4].

As seen on [4], problem (1) can be stated with the two types of boundary condition (2) and (3), by imposing each condition on different parts of Γ . However, the version of the problem presented later on Section 1.2.2 has the homogeneous Dirichlet boundary condition.

Finally, the Laplace equation (1) can be formulated on two different ways: first order formulation and second order formulation. The second order formulation is the one presented already on (1), which involves second order derivatives. Now, if we set $u = \nabla p$, the problem can be stated [4] as

$$\begin{cases} \operatorname{div}(u) = -f \text{ in } \Omega, \\ u = \nabla p \text{ in } \Omega, \end{cases} \quad (4)$$

which is the first order formulation for the problem and, notice that it only involves first order derivatives of u and p .

On Section 2, we compare both formulations of Laplace equation (1) and (4). We use Lagrange finite elements to solve (1) and Raviart-Thomas finite elements to solve (4).

1.1.2. Navier-Stokes equations

In this section we revise incompressible Stokes and Navier-Stokes equations with Dirichlet boundary condition. First, incompressible Navier-Stokes equations consist on finding the velocity $u : \Omega \rightarrow \mathbb{R}^3$ and the pressure $p : \Omega \rightarrow \mathbb{R}$ that solve the system of equations (5) [8].

$$\begin{aligned} \frac{\partial u}{\partial t} + (u \cdot \nabla)u - \nu \Delta u + \nabla p &= f, \text{ in } \Omega, \\ \nabla \cdot u &= 0, \text{ in } \Omega, \\ u &= g, \text{ in } \Gamma, \end{aligned} \tag{5}$$

where $\Omega \subseteq \mathbb{R}^3$ is the spatial domain, $\Gamma = \partial\Omega$ the boundary of the domain, ν is called the kinematic viscosity (more information below), f is the forcing term (given function), and g is the specified Dirichlet boundary condition.

On the other hand, when removing the non-linear term $(u \cdot \nabla)u$ from the first equation, we get Stokes linear equations which is system of equations (6) [8].

$$\begin{aligned} \frac{\partial u}{\partial t} - \nu \Delta u + \nabla p &= f, \text{ in } \Omega, \\ \nabla \cdot u &= 0, \text{ in } \Omega, \\ u &= g, \text{ in } \Gamma. \end{aligned} \tag{6}$$

Take into account that $u(x, y, z, t) = (u_x, u_y, u_z)$ is a vector for each point in time-space and that $p(x, y, z, t) = p$ is a scalar for each point in time-space. For real life fluid problems modeled by these equations, (x, y, z) is the position of a fluid's particle in space and t is the time. Also, the equation $\nabla \cdot u = 0$ is the one that establishes the incompressible condition for the fluid.

Kinematic Viscosity

On the modeling of fluid flow, as presented on [7], a very important parameter appears and its called *Reynolds number*, Re . On [7], it is defined as

$$Re = \frac{\rho L U}{\mu},$$

where ρ is the density of the fluid, L is the characteristic linear dimension of the domain of the flow, U is some characteristic velocity, and μ is the fluid's viscosity. For example, as seen on [7], L is defined supposing that $\Omega = [0, L]^n$, i.e., L is the length of the domain on each direction, and U can be taken as the square root of the average initial kinetic energy in Ω .

However, for the purpose of this work we neglect the parameters L and U in such way that $Re = \frac{\rho}{\mu}$. Therefore, the kinematic viscosity of a fluid is,

according to [7],

$$\nu := \frac{\mu}{\rho} = \frac{1}{Re}. \quad (7)$$

Notice that if the viscosity of the fluid, μ , is higher and the density of the fluid, ρ , is lower, then, the kinematic viscosity of the fluid, ν , is higher. This parameter quantifies the resistance that a fluid imposes to movement due to an external force, like gravity [7].

On Section 3, we do some numerical experiments in 2D and 3D using the Navier Miniapp of MFEM library, which solves (5), by using the formulation presented on Section 1.2.5.

1.2. The Finite Element Method

First, on Section 1.2.1 we show how the solution for a differential equation is also a solution for a minimization problem and a variational problem. This serves as a basic case for showing that partial differential equations are, in fact, solved via minimization or variational problems; which are the ones solved with finite element methods.

Then, in Sections 1.2.2 and 1.2.4 we study two finite element methods. On both of them, the following procedure was applied:

1. Consider the problem of solving Laplace equation (23) with homogeneous Dirichlet boundary condition.
2. Multiply by some function (*test function*) and integrate by parts. Apply boundary conditions.
3. Discretize the domain and select finite-dimensional function spaces for the solution and the test functions.
4. Produce a matrix system to solve for solution weights in the linear combination representation of the approximated solution.

The basis functions that form part of the finite-dimensional spaces are called *shape functions*. In Lagrange formulation, those are the functions in V_h , and in mixed formulation, those are the functions in H_h^k and L_h^k , where the parameter h denotes the size of the elements in the triangulation of the domain. Moreover, in Lagrange formulation the boundary condition is essential, and in mixed formulation, it is natural.

Furthermore, on Section 1.2.3 we show high order shape functions used in Lagrange finite element method and, on Section 1.2.5 we study the most common finite elements used to approximate the solution for Navier-Stokes equations.

1.2.1. FEM for elliptic problems

Let $\mathbb{D}$ be the two-point boundary value problem (8) taken from [2].

$$\begin{aligned} -u''(x) &= f(x), \quad x \in (0, 1), \\ u(0) &= u(1) = 0, \end{aligned} \tag{8}$$

where f is a given continuous function and $u''(x) = \frac{d^2 u(x)}{dx^2}$. Notice that this problem is a partial differential equation of the form presented on section 1.1 but with all the functions being of the form $\mathbb{R} \rightarrow \mathbb{R}$ and $\Omega = (0, 1)$ being a 1D domain, with homogeneous boundary condition (0 on $\Gamma = \{0, 1\}$).

By integrating $-u''(x) = f(x)$ twice, it is clear that the problem (8) has a unique solution u [2]. For example, if $f(x) = e^x$

$$\begin{aligned} -u''(x) &= e^x \\ \implies -u'(x) &= e^x + c_1 \\ \implies -u(x) &= e^x + c_1 x + c_2 \end{aligned}$$

and by applying boundary conditions,

$$\begin{cases} u(0) = 0 \implies c_2 = -1, \\ u(1) = 0 \implies c_1 = 1 - e. \end{cases}$$

It follows that $u(x) = -e^x + (e - 1)x + 1$ is the unique solution. Recall that the boundary conditions force the problem to have a unique solution, as mentioned previously on the work.

Now, following [2], define the linear space V of all continuous functions on $[0, 1]$ that vanish at $\{0, 1\}$, whose derivative is piecewise continuous and bounded on $[0, 1]$. Also, define the linear functional $F : V \rightarrow \mathbb{R}$, by

$$F(v) = \frac{1}{2} \int_0^1 [v'(x)]^2 dx - \int_0^1 f(x)v(x)dx.$$

With this settled, let $\mathbb{M}$ be the optimization problem of finding $u \in V$ such that

$$F(u) \leq F(v) \tag{9}$$

for all $v \in V$, and let $\mathbb{V}$ be the variational problem of finding $u \in V$ such that

$$\int_0^1 u'(x)v'(x)dx = \int_0^1 f(x)v(x)dx \tag{10}$$

for all $v \in V$.

Remark that if $u, v \in V$ then $w = u + \alpha v \in V$ for any $\alpha \in \mathbb{R}$, because w

is continuous in $[0, 1]$; $w(1) = w(0) = u(0) + \alpha v(0) = 0$, i.e. vanishes on $\{0, 1\}$; and $w' = u' + \alpha v'$ is piecewise continuous and bounded on $[0, 1]$. The rest of this section is focused on showing that $\mathbb{D}$, $\mathbb{M}$ and $\mathbb{V}$ are equivalent problems [2].

Equivalence $\mathbb{D} \iff \mathbb{V}$

Let $u_{\mathbb{D}}$ be the solution for $\mathbb{D}$. Then, multiply $-u_{\mathbb{D}}''(x) = f(x)$ on both sides by some $v \in V$ (this v is called a test function) and integrate to obtain

$$-\int_0^1 u_{\mathbb{D}}''(x)v(x)dx = \int_0^1 f(x)v(x)dx. \quad (11)$$

Using the formula for integration by parts,

$$\int_0^1 a(x)b'(x)dx = a(1)b(1) - a(0)b(0) - \int_0^1 a'(x)b(x)dx, \quad (12)$$

with $a(x) = v(x)$ and $b(x) = u_{\mathbb{D}}'(x)$, and applying the fact that $v(0) = v(1) = 0$ we get

$$\int_0^1 u_{\mathbb{D}}''(x)v(x)dx = -\int_0^1 v'(x)u_{\mathbb{D}}'(x)dx. \quad (13)$$

Then, replacing (13) on equation (11) we get that

$$\int_0^1 v'(x)u_{\mathbb{D}}'(x)dx = \int_0^1 f(x)v(x)dx. \quad (14)$$

Notice that (14) is the equation associated to the variational problem $\mathbb{V}$ (see (10)), and as $v \in V$ was arbitrary, we have that $u_{\mathbb{D}}$ satisfies (14) (and so, (10)) for all $v \in V$. Therefore, $u_{\mathbb{D}}$ is also a solution for $\mathbb{V}$. ✓

On the other hand, let $u_{\mathbb{V}} \in V$ be the solution for $\mathbb{V}$ (on [2], it is shown that the solution for $\mathbb{V}$ is unique). Then, we have by (10) that, for all $v \in V$,

$$\int_0^1 u_{\mathbb{V}}'(x)v'(x)dx - \int_0^1 f(x)v(x)dx = 0. \quad (15)$$

Now, applying integration by parts (12) in the same way as before, we obtain (13). Replacing (13) on (15) and unifying the integral, we get

$$-\int_0^1 [u_{\mathbb{V}}''(x) + f(x)]v(x)dx = 0 \quad (16)$$

for all $v \in V$. Then, by (18) we have that $u_{\mathbb{V}}''(x) + f(x) = 0$ for $x \in (0, 1)$. In other words,

$$-u_{\mathbb{V}}''(x) = f(x), \quad x \in (0, 1). \quad (17)$$

Notice that (17), along with the fact that $u_{\mathbb{V}}(0) = u_{\mathbb{V}}(1) = 0$, is the equation associated to the differential problem $\mathbb{D}$ (see (8)). Therefore, $u_{\mathbb{V}}$ is also

a solution for $\mathbb{V}$ as long as $u''(x)$ exists and is continuous (regularity assumption). But, this last condition for $u_{\mathbb{V}}$ holds, as seen on [2], so, the result holds. ✓

To complete the proof, we have to prove (18), which is exercise 1.1 from [2]:

$$\begin{aligned} &\text{If } w \text{ is continuous on } [0, 1] \text{ and} \\ &\int_0^1 w(x)v(x)dx = 0 \text{ for all } v \in V, \\ &\text{then } w(x) = 0 \text{ for all } x \in (0, 1). \end{aligned} \quad (18)$$

Proof: By contradiction, suppose that $w(x) \neq 0$ for some $x \in (0, 1)$. Then, $w(x_0) = c \in \mathbb{R}$ for $x_0 \in (0, 1)$. Without loss of generality, suppose that $c > 0$ (for $c < 0$ the argument is analogous). As w is continuous, there is an interval centered at x_0 , $I = (x_0 - \delta, x_0 + \delta)$, such that $f(x) > 0$ for $x \in I$. Now, define

$$v(x) = \begin{cases} 0, & \text{if } x \notin I, \\ \frac{c}{\delta}[x + (\delta - x_0)], & \text{if } x \in (x_0 - \delta, x_0) \\ -\frac{c}{\delta}[x - (\delta + x_0)], & \text{if } x \in (x_0, x_0 + \delta), \end{cases}$$

which is a function sketched on Figure 1.

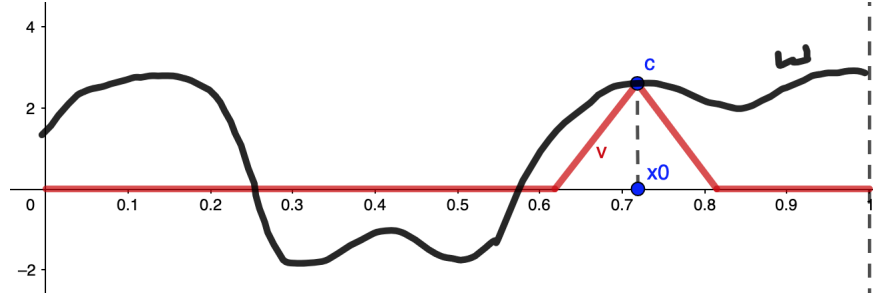


Figure 1: Sketch of the proof for (18) involving an arbitrary function $w(x)$ (solid black line) and the built function $v \in V$ (solid red line).

Notice that $v(0) = v(1) = 0$ because there is always a $\delta > 0$ such that $0 < x_0 - \delta$ and $x_0 + \delta < 1$, and $v(x) = 0$ for $x \notin I$. Also, $v(x)$ is continuous on $[0, 1]$ by construction (it was built with 4 lines that have connections in $(x_0 - \delta, 0)$, (x_0, c) and $(x_0 + \delta, 0)$). Moreover,

$$v'(x) = \begin{cases} 0, & \text{if } x \notin I, \\ \frac{c}{\delta}, & \text{if } x \in (x_0 - \delta, x_0), \\ -\frac{c}{\delta}, & \text{if } x \in (x_0, x_0 + \delta), \end{cases}$$

is clearly piecewise continuous and bounded on $[0, 1]$. Therefore, $v \in V$.

However, as $w(x)v(x) > 0$ for $x \in I$ and $w(x)v(x) = 0$ for $x \notin I$, then

$$\int_0^1 w(x)v(x)dx = \int_{x_0-\delta}^{x_0+\delta} w(x)v(x)dx > 0.$$

In other words, if $w(x) \neq 0$ for some $x \in (0, 1)$, then we found $v \in V$ such that $\int_0^1 w(x)v(x)dx \neq 0$, which contradicts the hypothesis that $\int_0^1 w(x)v(x)dx = 0$ for all $v \in V$. Therefore, $w(x) = 0$ for all $x \in (0, 1)$. $\square$

Equivalence $\mathbb{V} \iff \mathbb{M}$

Let $u_{\mathbb{V}} \in V$ be the solution for $\mathbb{V}$. Take some $v \in V$ and set $w = v - u_{\mathbb{V}} \in V$. Then

$$\begin{aligned} F(v) &= F(u + w) \\ &= \frac{1}{2} \int_0^1 [u'_{\mathbb{V}}(x) + w'(x)]^2 dx - \int_0^1 f(x)[u_{\mathbb{V}}(x) + w(x)]dx \\ &= \frac{1}{2} \left(\int_0^1 [u'_{\mathbb{V}}(x)]^2 dx + \int_0^1 2u'_{\mathbb{V}}(x)w'(x)dx + \int_0^1 [w'(x)]^2 dx \right) \\ &\quad - \int_0^1 f(x)u_{\mathbb{V}}(x)dx - \int_0^1 f(x)w(x)dx \\ &= \frac{1}{2} \int_0^1 [u'_{\mathbb{V}}(x)]^2 dx - \int_0^1 f(x)u_{\mathbb{V}}(x)dx + \int_0^1 u'_{\mathbb{V}}(x)w'(x)dx \\ &\quad - \int_0^1 f(x)w(x)dx + \frac{1}{2} \int_0^1 [w'(x)]^2 dx \\ &\stackrel{(14)}{=} \frac{1}{2} \int_0^1 [u'_{\mathbb{V}}(x)]^2 dx - \int_0^1 f(x)u_{\mathbb{V}}(x)dx + \frac{1}{2} \int_0^1 [w'(x)]^2 dx \\ &= F(u_{\mathbb{V}}) + \frac{1}{2} \int_0^1 [w'(x)]^2 dx \geq F(u_{\mathbb{V}}). \end{aligned}$$

In other words, we have that

$$F(v) \geq F(u_{\mathbb{V}}). \tag{19}$$

Notice that (19) is the equation associated to the optimization problem $\mathbb{M}$ (see (9)), and as $v \in V$ was arbitrary, we have that $u_{\mathbb{V}}$ satisfies (19) (and so, (9)) for all $v \in V$. Therefore, $u_{\mathbb{V}}$ is also a solution for $\mathbb{M}$. $\checkmark$

On the other hand, let $u_{\mathbb{M}} \in V$ be a solution for $\mathbb{M}$. Then, for any $v \in V$ and any $\alpha \in \mathbb{R}$ we have that $u_{\mathbb{M}} + \alpha v \in V$, and so $F(u_{\mathbb{M}}) \leq F(u_{\mathbb{M}} + \alpha v)$ by (9). Therefore, the minimum for $F(u_{\mathbb{M}} + \alpha v)$ is achieved at $\alpha = 0$.

Now, define $g(\alpha) = F(u_{\mathbb{M}} + \alpha v)$, which is a differentiable function [2]. We have that

$$\begin{aligned} g(\alpha) &= F(u_{\mathbb{M}} + \alpha v) \\ &= \frac{1}{2} \int_0^1 [u'_{\mathbb{M}}(x) + \alpha v'(x)]^2 dx - \int_0^1 f(x)[u_{\mathbb{M}}(x) + \alpha v(x)] dx \\ &= \frac{1}{2} \int_0^1 [u'_{\mathbb{M}}(x)]^2 dx + \alpha \int_0^1 u'_{\mathbb{M}}(x)v'(x) dx + \frac{\alpha^2}{2} \int_0^1 [v'(x)]^2 dx \\ &\quad - \int_0^1 f(x)u_{\mathbb{M}}(x) dx - \alpha \int_0^1 f(x)v(x) dx. \end{aligned} \quad (20)$$

Since $g(\alpha) = F(u_{\mathbb{M}} + \alpha v)$ has a minimum at $\alpha = 0$, then $g'(0) = 0$. Therefore, taking the derivative on (20) and replacing $\alpha = 0$, we get

$$\begin{aligned} g'(\alpha) &= \int_0^1 u'_{\mathbb{M}}(x)v'(x) dx + \alpha \int_0^1 [v'(x)]^2 dx - \int_0^1 f(x)v(x) dx \\ \implies g'(0) &= \int_0^1 u'_{\mathbb{M}}(x)v'(x) dx - \int_0^1 f(x)v(x) dx. \end{aligned} \quad (21)$$

Finally, as $g'(0) = 0$, we get from (21), that

$$\int_0^1 u'_{\mathbb{M}}(x)v'(x) dx = \int_0^1 f(x)v(x) dx. \quad (22)$$

Notice that (22) is the equation associated to the variational problem $\mathbb{V}$ (see (10)), and as $v \in V$ was arbitrary, we have that $u_{\mathbb{M}}$ satisfies (22) (and so, (10)) for all $v \in V$. Therefore, $u_{\mathbb{M}}$ is also a solution for $\mathbb{V}$. ✓

As mentioned before, on [2], it is shown that the solution for $\mathbb{V}$ is unique. Therefore, as $\mathbb{V}$ and $\mathbb{M}$ are equivalent, then the solution for $\mathbb{M}$ is also unique.

1.2.2. Lagrange finite elements

In this section we consider the Laplace equation [2],

$$\begin{aligned} -\Delta p &= f \text{ in } \Omega, \\ p &= 0 \text{ in } \Gamma, \end{aligned} \quad (23)$$

where $\Omega \subseteq \mathbb{R}^2$ is an open-bounded domain with boundary Γ , f is a given function and $\Delta p = \frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2}$. Following [2], consider the space

$$V = \{v : v \text{ continuous on } \Omega, \frac{\partial v}{\partial x}, \frac{\partial v}{\partial y} \text{ piecewise continuous on } \Omega \text{ and } v = 0 \text{ on } \Gamma\}.$$

Alternatively we can work in the Sobolev space (see [2–4])

$$H^1(\Omega) = \{v \in L^2(\Omega) \mid \frac{\partial v}{\partial x}, \frac{\partial v}{\partial y} \in L^2(\Omega)\}.$$

Here $L^2(\Omega) = \{v : \Omega \rightarrow \mathbb{R} \mid \int_{\Omega} v^2 < \infty\}$.

We multiply the first equation of (23) by some $v \in V$ (referred to as *test function*) and integrate over Ω to obtain

$$-\int_{\Omega} \Delta p \, v = \int_{\Omega} f \, v. \quad (24)$$

Applying divergence theorem we obtain the Green's formula ([2]),

$$-\int_{\Omega} \Delta p \, v = \int_{\Omega} \nabla v \cdot \nabla p - \int_{\Gamma} v \, \nabla p \cdot \eta, \quad (25)$$

where η is the outward unit normal to Γ . Since $v = 0$ on Γ , the third integral equals 0. Note that the boundary integral does not depend on p 's value on Γ but rather on the normal derivative of p in Γ . Due to this fact the boundary condition $p = 0$ on Γ is known as an **essential** boundary condition.

Then, replacing (25) on (24), we get,

$$\int_{\Omega} \nabla v \cdot \nabla p = \int_{\Omega} f \, v. \quad (26)$$

This holds for all $v \in V$. This is called weak formulation of the Laplace equation (23). We remark that, according to [2], if $p \in V$ satisfies (26) for all $v \in V$ and is sufficiently regular, then p also satisfies (23), i.e., it's a (classical) solution for our problem. For more details see [2] and references therein.

In order to set the problem for a computer to solve it, we are going to discretize it and encode it into a linear system.

First, consider a *triangulation* T_h of the domain Ω . This is, $T_h = \{K_1, \dots, K_m\}$ a set of non-overlapping triangles such that $\Omega = K_1 \cup \dots \cup K_m$ and no vertex (N_i) of one triangle lies on the edge of another triangle, as seen on Figure 2.

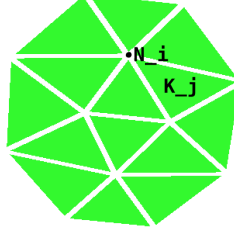


Figure 2: A triangulation for a given domain Ω showing a node N_i , and formed by some triangles K_j . *Note:* Triangles have been separated in the edges to take a better look, but the triangulation has no empty spaces. Visualization: [6].

The h in the notation T_h is a measure of the size of mesh, it usually refers to a typical element diameter or perhaps to the largest element diameter in the triangulation. In this manuscript h is defined by $h = \max\{\text{diam}(K) : K \in T_h\}$ where $\text{diam}(K)$ = longest side of K .

Now, let $V_h = \{v : v \text{ continuous on } \Omega, v|_K \text{ linear for } K \in T_h, v = 0 \text{ on } \Gamma\}$. We consider the nodes $(N_1, \dots, N_M)$ of the triangulation that are not on the boundary, because $p = 0$ there, and we define some functions $\varphi_j \in V_h$ in such way that

$$\varphi_j(N_i) = \begin{cases} 1 & , i = j \\ 0 & , i \neq j \end{cases}$$

for $i, j = 1, \dots, M$. See Figure 3 for an illustration of $\varphi_j \in V_h$.

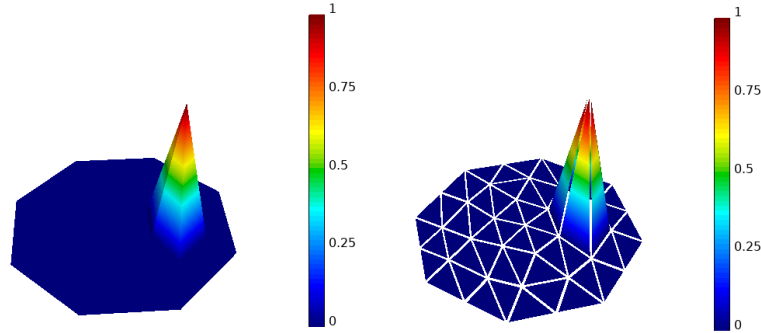


Figure 3: Illustration of the function φ_j produced with MFEM library. On the left picture we plot the function φ_j . On the right picture we show the same plot depicting the elements of the underlying triangulation. Visualization: [6].

With this, $V_h = \text{span}\{\varphi_i : i = 1, \dots, M\}$ and for any given $v \in V_h$ we have $v(x) = \sum_{j=1}^M \xi_j \varphi_j(x)$, with $\xi_j = v(N_j)$ and $x \in \Omega \cup \Gamma$. So, V_h is a finite-dimensional subspace of V . See [2] for details.

Then, if $p_h \in V_h$ satisfies (26) for all $v \in V_h$, in particular,

$$\int_{\Omega} \nabla p_h \cdot \nabla \varphi_j = \int_{\Omega} f \varphi_j, \quad j = 1, \dots, M. \quad (27)$$

Since $\nabla p_h = \sum_{i=1}^M \xi_i \nabla \varphi_i$ with $\xi_i = p_h(N_i)$, replacing on (27) we get,

$$\sum_{i=1}^M \xi_i \int_{\Omega} \nabla \varphi_i \cdot \nabla \varphi_j = \int_{\Omega} f \varphi_j, \quad j = 1, \dots, M. \quad (28)$$

Finally, (28) is a linear system of M equations and M unknowns $(\xi_1, \dots, \xi_M)$, which can be written as,

$$A\xi = b, \quad (29)$$

where $A[i, j] = \int_{\Omega} \nabla \varphi_i \cdot \nabla \varphi_j$, $\xi[i] = p_h(N_i)$ and $b[i] = \int_{\Omega} f \varphi_i$.

We can solve (29) with MFEM library as done on Section 2 (Example#1). Before continuing with the next section, let us show some theorems regarding the error between the solution p for problem (23) and its approximation p_h . The theorems are presented on the general form but, after the proof, we show how is it used on our particular problem.

For the following theorem, A is a bilinear form on $V \times V$ and L is a linear form on V such that

1. **A is continuous ($\mathcal{C}$)**

There is a constant $\gamma > 0$ such that

$$|A(v, w)| \leq \gamma \|v\|_V \|w\|_V, \quad \forall v, w \in V.$$

2. **A is V -elliptic (V_{ϵ})**

There is a constant $\alpha > 0$ such that

$$\alpha \|v\|_V^2 \leq A(v, v).$$

Theorem 1.1. [2] **Céa Lemma**

If $p \in V$ is the solution for

$$A(p, v) = L(v), \quad \forall v \in V$$

and $p_h \in V_h \subset V$ is the solution for

$$A(p_h, v_h) = L(v_h), \quad \forall v_h \in V_h$$

then,

$$\|p - p_h\|_V \leq \frac{\gamma}{\alpha} \|p - v_h\|_V, \quad \forall v_h \in V_h.$$

Proof. Using the hypothesis that $A(p, v) = L(v)$ for all $v \in V$, along with the fact that $V_h \subset V$, we have that $A(p, v_h) = L(v_h)$ for all $v_h \in V_h$. Now, we subtract the last equation with the one given as hypothesis, $A(p_h, v_h) = L(v_h)$, to get, $A(p - p_h, v_h) = L(v_h) - L(v_h) = 0$ for all $v_h \in V_h$.

For an arbitrary $w \in V_h$, let $v_h = p_h - w \in V_h$. Then,

$$\begin{aligned} & \alpha \|p - p_h\|_V^2 \\ & \leq A(p - p_h, p - p_h) + 0 \quad (\text{See } V_\epsilon) \\ & = A(p - p_h, p - p_h) + A(p - p_h, w) \\ & = A(p - p_h, p - p_h + w) \\ & = A(p - p_h, p - p_h + p_h - v_h) \\ & = A(p - p_h, p - v_h) \\ & \leq \gamma \|p - p_h\|_V \|p - v_h\|_V \quad (\text{See } \mathcal{C}). \end{aligned}$$

In other words, we have that $\alpha \|p - p_h\|_V^2 \leq \gamma \|p - p_h\|_V \|p - v_h\|_V$ for all $v_h \in V_h$. Dividing by $\alpha \|p - p_h\|_V$ on both sides, we get, $\|p - p_h\|_V \leq \frac{\gamma}{\alpha} \|p - v_h\|_V$ for all $v_h \in V_h$. Note that $\|p - p_h\|_V \neq 0$ because p_h is supposed to be an approximation for p , and not the exact solution. $\square$

On the particular case of (26), we have that $A(p, v) = \int_\Omega \nabla v \cdot \nabla p$ and $L(v) = \int_\Omega f v$. Using Cauchy-Schwarz Inequality for Integrals we have that

$$A(p, v) = \int_\Omega \nabla v \cdot \nabla p \leq \sqrt{\int_\Omega |\nabla p|^2} \cdot \sqrt{\int_\Omega |\nabla v|^2} = 1 \cdot \|p\|_V \|v\|_V.$$

In other words, the parameter for the continuity of the bilinear form of our particular case is $\gamma = 1$. Also, notice that

$$1 \cdot \|v\|_V^2 = \left(\sqrt{\int_\Omega |\nabla v|^2} \right)^2 = \int_\Omega |\nabla v|^2 = \int_\Omega \nabla v \nabla v = A(v, v).$$

That is, the parameter for the V -ellipticity of the bilinear form for our particular case is $\alpha = 1$. Therefore, Céa Lemma ensures that $\sqrt{\int_\Omega |p - p_h|^2} \leq \sqrt{\int_\Omega |p - v_h|^2}$ for all $v_h \in V_h$.

Now, before presenting the second theorem, define the operator $\mathcal{I}^h : \mathcal{C}(\Omega) \rightarrow V_h$ that associates every continuous function whose domain is Ω , $f \in \mathcal{C}(\Omega)$, with a function $\mathcal{I}^h f \in V_h$ [4]. This operator is an interpolation operator defined by the nodes of the triangulation of Ω : if $T_h = \{K_1, \dots, K_m\}$ is a triangulation of Ω , then $\mathcal{I}^h f(K_i) = f(K_i)$, $i = 1, \dots, m$.

Theorem 1.2. [4] *Let $\Omega \subseteq \mathbb{R}^2$ be a polygonal domain. Let $\{T_{h_i}\}$ be a family of triangulations of Ω , with T_{h_i} being a quasi-uniform triangulation. Then,*

$$\|\mathcal{I}^h p - p\|_1 \leq c \cdot h \cdot \|p\|_2,$$

where

$$\|f\|_2 = \left(\int_{\Omega} f(x)^2 + |\nabla f(x)|^2 + \sum_{ij} (\partial_{ij} f)^2 dx \right)^{1/2}.$$

The proof of this theorem is out of the scope for this work. However, it can be checked on [4]. In summary, for our case, Céa Lemma states that Laplace problem can be approximated by the space V_h , and Theorem 1.2 states that the approximation is a good one.

1.2.3. Lagrange spaces of higher order

This short section has the purpose of explaining Lagrange finite element spaces of higher order. Previously, on Section 1.2.2, when introducing Lagrangian elements, the shape function's degree was set to one. Better approximations can be obtained by using polynomials of higher order. One can define, for a fixed order k ,

$$V_h^k = \{v : v \text{ continuous on } \Omega, \\ v|_K \text{ polynomial of order at most } k, K \in T_h, v = 0 \text{ on } \Gamma\}.$$

For example, as seen in [4], the space of Bell triangular finite elements for a given triangulation T_h is the space of functions that are polynomials of order 5 when restricted to every triangle $K \in T_h$. That is, if v is in this space, then,

$$v|_K(x, y) = a_1 x^5 + a_2 y^5 + a_3 x^4 y + a_4 x y^4 + \cdots + a_{16} x + a_{17} y + a_{18}$$

for all $K \in T_h$. Here, the constants a_i , $i = 1, \dots, 18$ correspond to v 's *DOF* (degrees of freedom).

On Figures 4 and 5, we present a visualization of some shape functions of different orders. We encourage the reader to compare them with Figure 3 and notice the degree of the polynomial in the nonzero part of the shape functions.

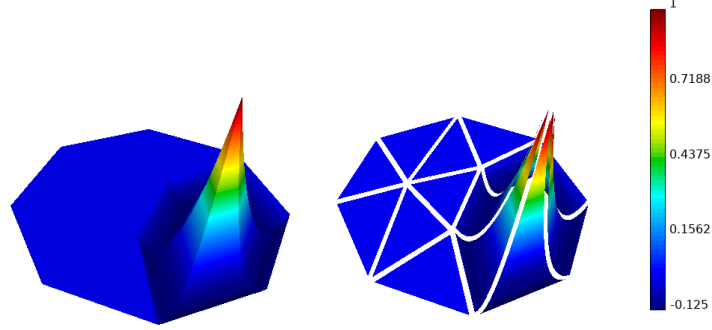


Figure 4: Illustration of finite element basis (shape) functions of order 2. On the left picture we show one continuous basis function. On the right we also show the underlying triangulation. Visualization: [6].

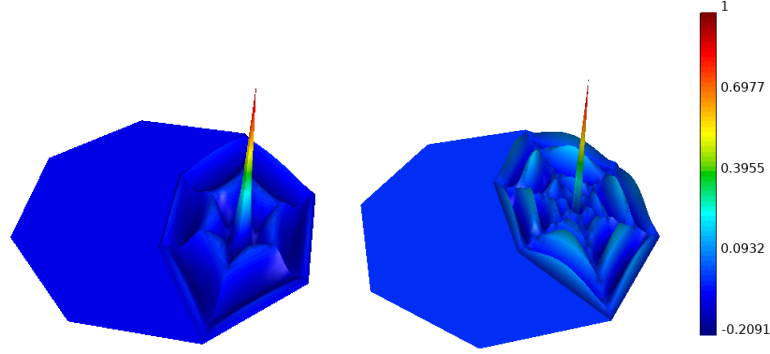


Figure 5: Illustration of finite elements basis function of orders 5 (left) and 10 (right). Visualization: [6].

1.2.4. Raviart-Thomas finite elements

First, let's define some important spaces, where Ω is a bounded domain in $\mathbb{R}^2$ and Γ its boundary. See [2–4] and references therein for details. The space of all square integrable functions,

$$L^2(\Omega) = \{v : \Omega \rightarrow \mathbb{R} \mid \int_{\Omega} v^2 < \infty\}.$$

We also use the first order Sobolev space,

$$H^1(\Omega) = \{v \in L^2(\Omega) \mid \frac{\partial v}{\partial x}, \frac{\partial v}{\partial y} \in L^2(\Omega)\}$$

and the subspace of $H^1(\Omega)$ of functions with vanishing value on the boundary,

$$H_0^1(\Omega) = \{v \in H^1(\Omega) \mid v = 0 \text{ on } \Gamma\}.$$

We also introduce the space of square integrable vector functions with square integrable divergence,

$$H(\text{div}; \Omega) = \{\mathbf{v} \in L^2(\Omega) \times L^2(\Omega) \mid \text{div}(\mathbf{v}) \in L^2(\Omega)\}.$$

As above, let $\Omega \in \mathbb{R}^2$ be a bounded domain with boundary Γ and consider problem (23). This time we require explicitly that $f \in L^2(\Omega)$. Recall (from Section 1.2.2) that this problem can be reduced to

$$\int_{\Omega} \nabla v \cdot \nabla p = \int_{\Omega} f v, \text{ for all } v \in V,$$

where Dirichlet boundary condition ($p = 0$ in Γ) is essential. Recall that we can take $V = H_0^1(\Omega)$ as seen in [2, 3].

Let $u = \nabla p$ in Ω . Then, problem (23) can be written as the following system of first order partial differential equations,

$$\begin{aligned} u &= \nabla p \text{ in } \Omega \\ \text{div}(u) &= -f \text{ in } \Omega \\ p &= 0 \text{ in } \Gamma, \end{aligned} \tag{30}$$

because $\Delta p = \text{div}(\nabla p)$. Now, following a similar procedure as in Section 1.2.2, multiply the first equation of (30) by some $\mathbf{v} \in H(\text{div}; \Omega)$ and integrate both sides to obtain,

$$\int_{\Omega} u \cdot \mathbf{v} = \int_{\Omega} \nabla p \cdot \mathbf{v}. \tag{31}$$

Consider Green's identity [3],

$$\int_{\Omega} \mathbf{v} \cdot \nabla p + \int_{\Omega} p \text{div}(\mathbf{v}) = \int_{\Gamma} (\mathbf{v} \cdot \eta) p, \tag{32}$$

where η is the normal vector exterior to Γ .

Replacing (32) in (31), and considering the third equation of (30), we get,

$$\int_{\Omega} u \cdot \mathbf{v} + \int_{\Omega} p \text{div}(\mathbf{v}) = \int_{\Gamma} (\mathbf{v} \cdot \eta) p, \tag{33}$$

On the other hand, we can multiply the second equation of problem (30) by some $w \in L^2(\Omega)$, integrate and obtain,

$$\int_{\Omega} w \text{div}(u) = - \int_{\Omega} f w. \tag{34}$$

Note that the boundary integral depends directly on the value of p in Γ . And, this is referred to as the case of a **natural** boundary condition. Observe that the same boundary condition appeared as an essential boundary condition in the second order formulation considered before (Section 1.2.2). In this first order formulation it showed up as a natural boundary condition.

Finally, applying boundary condition $p = 0$ in Γ into (33), and joining (33) and (34). We get the following problem deduced from (30),

$$\begin{aligned} \int_{\Omega} u \, \mathbf{v} + \int_{\Omega} p \, \operatorname{div}(\mathbf{v}) &= 0 \\ \int_{\Omega} w \, \operatorname{div}(u) &= - \int_{\Omega} f \, w. \end{aligned} \quad (35)$$

For this problem, which is a variational formulation of (30), the objective is to find $(u, p) \in H(\operatorname{div}; \Omega) \times L^2(\Omega)$ such that it is satisfied for all $\mathbf{v} \in H(\operatorname{div}; \Omega)$ and all $w \in L^2(\Omega)$.

For the discretized problem related to (35), in [3] the following spaces are defined for a *triangulation* T_h of the domain Ω and a fixed integer $k \geq 0$,

$$\begin{aligned} H_h^k &:= \{\mathbf{v}_h \in H(\operatorname{div}; \Omega) \mid \mathbf{v}_h|_K \in RT_k(K) \text{ for all } K \in T_h\}, \text{ and} \\ L_h^k &:= \{w_h \in L^2(\Omega) \mid w_h|_K \in \mathbb{P}_k(K) \text{ for all } K \in T_h\}, \end{aligned}$$

where

$$\begin{aligned} \mathbb{P}_k(K) &= \{p : K \rightarrow \mathbb{R} \mid p \text{ is a polynomial of degree } \leq k\}, \text{ and} \\ RT_k(K) &= [\mathbb{P}_k(K) \times \mathbb{P}_k(K)] + \mathbb{P}_k(K)x. \end{aligned}$$

Note that $\mathbf{p} \in RT_k(K)$ if and only if there are some $p_0, p_1, p_2 \in \mathbb{P}_k(K)$ such that

$$\mathbf{p}(x) = \begin{pmatrix} p_1(x) \\ p_2(x) \end{pmatrix} + p_0(x) \begin{pmatrix} x \\ y \end{pmatrix} \text{ for all } \begin{pmatrix} x \\ y \end{pmatrix} \in K,$$

and, also note that $\mathbf{p}$ has a degree of $k + 1$.

Then, (35) gives the following discrete problem: find $(u_h, p_h) \in H_h^k \times L_h^k$ such that

$$\begin{aligned} \int_{\Omega} u_h \, \mathbf{v}_h + \int_{\Omega} p_h \, \operatorname{div}(\mathbf{v}_h) &= 0 \\ \int_{\Omega} w_h \, \operatorname{div}(u_h) &= - \int_{\Omega} f \, w_h, \end{aligned} \quad (36)$$

for all $\mathbf{v}_h \in H_h^k$ and all $w_h \in L_h^k$.

As spaces H_h^k and L_h^k are finite dimensional, they have a finite basis. That is, $H_h^k = \operatorname{span}\{\varphi_i : i = 1, \dots, M\}$ and $L_h^k = \operatorname{span}\{\psi_j : j = 1, \dots, N\}$. Then, $u_h = \sum_{i=1}^M u_i \varphi_i$ and $p_h = \sum_{j=1}^N p_j \psi_j$, where u_i and p_j are scalars.

In particular, as $\varphi_k \in H_h^k$ and $\psi_l \in L_h^k$, we have that problem (36) can be written as,

$$\begin{aligned} \int_{\Omega} \left(\sum_{i=1}^M u_i \varphi_i \right) \varphi_k + \int_{\Omega} \left(\sum_{j=1}^N p_j \psi_j \right) \operatorname{div}(\varphi_k) &= 0 \\ \int_{\Omega} \psi_l \operatorname{div} \left(\sum_{i=1}^M u_i \varphi_i \right) &= \int_{\Omega} f \psi_l, \end{aligned} \quad (37)$$

for $k = 1, \dots, M$ and $l = 1, \dots, N$. Which is equivalent to the following, by rearranging scalars,

$$\begin{aligned} \sum_{i=1}^M u_i \int_{\Omega} \varphi_i \cdot \varphi_k + \sum_{j=1}^N p_j \int_{\Omega} \psi_j \operatorname{div}(\varphi_k) &= 0 \\ \sum_{i=1}^M u_i \int_{\Omega} \psi_l \operatorname{div}(\varphi_i) &= \int_{\Omega} f \psi_l, \end{aligned} \quad (38)$$

for $k = 1, \dots, M$ and $l = 1, \dots, N$. The problem (38) can be formulated into the following matrix system

$$\begin{pmatrix} A & B \\ B^t & 0 \end{pmatrix} \begin{pmatrix} U \\ P \end{pmatrix} = \begin{pmatrix} 0 \\ F \end{pmatrix}, \quad (39)$$

where A is a $N \times N$ matrix, B is a $M \times N$ matrix with B^t its transpose, U is a M -dimensional column vector and P, F are N -dimensional column vectors. The entries of these arrays are $A[i, j] = \int_{\Omega} \varphi_i \cdot \varphi_j$, $B[i, j] = \int_{\Omega} \psi_j \operatorname{div}(\varphi_i)$, $U[i] = u_i$, $P[i] = p_i$ and $F[i] = \int_{\Omega} f \psi_i$.

The linear system (39) can be solved for (U, P) with a computer using MFEM library. Note that with the entries of U and P , the solution (u_h, p_h) of (36) can be computed by their basis representation.

The spaces defined to discretize the problem are called Raviart-Thomas finite element spaces. The fixed integer k is also called the order of the shape functions or the order of the finite element space. The parameter h is the same as in Section 1.2.2, which is a measure of size for T_h . See [3] for details.

1.2.5. Taylor-Hood finite elements

In this section, we show the spatial discretization done in [8], for Stokes equations (40),

$$\begin{aligned} \frac{\partial u}{\partial t} - \nu \Delta u + \nabla p &= f, \text{ in } \Omega, \\ \nabla \cdot u &= 0, \text{ in } \Omega, \\ u &= g, \text{ in } \Gamma, \end{aligned} \quad (40)$$

and then mention the corresponding spatial discretization for Navier-Stokes equations (41), as an extension of the previous one,

$$\begin{aligned} \frac{\partial u}{\partial t} + (u \cdot \nabla)u - \nu \Delta u + \nabla p &= f, \text{ in } \Omega, \\ \nabla \cdot u &= 0, \text{ in } \Omega, \\ u &= g, \text{ in } \Gamma. \end{aligned} \quad (41)$$

When applying a numerical method to solve both systems of equations, (40) and (41), time has to be discretized too. However, time discretization is out of the scope of this paper (it can be found on section 4 of [8]).

First of all, let $T_h = \{K_1, \dots, K_m\}$ be a *discretization* of the domain Ω . That is, $\Omega = K_1 \cup \dots \cup K_m$, and $K_1, \dots, K_m$ don't overlap between them, and no vertex of one of them lies on the edge of another (check triangulation on Section 1.2.2). If Ω is in 2D, K_i is a quadrilateral, and if Ω is in 3D, K_i is a hexahedron. Then, define the following finite element function spaces on T_h , where $d \in \{2, 3\}$ is the dimension of Ω [8].

$$\begin{aligned} U_h^k &= \{v \in (H^1(\Omega))^d : v(K) \in (\mathcal{Q}_k(K))^d \text{ for all } K \in T_h\}, \\ P_h^k &= \{s \in H^1(\Omega) : s(K) \in \mathcal{Q}_k(K) \text{ for all } K \in T_h\}, \end{aligned}$$

where $\mathcal{Q}_k(K)$ is the set of all polynomials whose degree on each of their variables is less or equal than k , with domain K . For example, if K is two dimensional, $\mathcal{Q}_2(K) = \{a_0 + a_1x + a_2y + a_3x^2 + a_4y^2 + a_5xy + a_6x^2y + a_7xy^2 + a_8x^2y^2 : a_i \in \mathbb{R}, i = 1, \dots, 8\}$. Notice that the total degree of $a_8x^2y^2$ is 4, but the degree on each variable (x or y) is just 2, as desired.

As done on previous sections, multiply the equations of (40) by some test function $v \in U_h^k$ and $s \in P_h^k$, respectively, to obtain

$$\begin{aligned} \int_{\Omega} \frac{\partial u}{\partial t} v - \nu \int_{\Omega} \Delta u v + \int_{\Omega} \nabla p \cdot v &= \int_{\Omega} f v, \\ \int_{\Omega} (\nabla \cdot u) s &= 0. \end{aligned} \quad (42)$$

Now, applying Green's formula (25) with homogeneous boundary condition ($u = g = 0$ in Γ), as done on Section 1.2.2, the term $-\nu \int_{\Omega} \Delta u v$ becomes

$\nu \int_{\Omega} \nabla u \cdot \nabla v$. Also, it is usual to multiply the second equation by -1 , therefore, (42) becomes the finite element formulation (43) found on [8]. The idea is to find $(u, p) \in (U_h^k, P_h^k)$, for all $(v, s) \in (U_h^k, P_h^k)$, such that

$$\begin{aligned} \int_{\Omega} \frac{\partial u}{\partial t} v - \nu \int_{\Omega} \nabla u \cdot \nabla v + \int_{\Omega} \nabla p \cdot v &= \int_{\Omega} f v, \\ - \int_{\Omega} (\nabla \cdot u) s &= 0. \end{aligned} \quad (43)$$

Let $\{\phi_i : i = 1, \dots, n\}$ be a basis for U_h^k and $\{\psi_j : j = 1, \dots, m\}$ be a basis for P_h^k . Therefore,

$$u(x, t) = \sum_{i=1}^n u_i(t) \phi_i(x)$$

and

$$p(x, t) = \sum_{j=1}^m p_j(t) \psi_j(x)$$

where u_i and p_j are functions depending only on t . Replacing these representations on (43) and noticing that $(\phi_i, \psi_j) \in (U_h^k, P_h^k)$, we get the system of equations (44), with $I = 1, \dots, n$ and $J = 1, \dots, m$.

$$\begin{aligned} \int_{\Omega} \frac{\partial (\sum_{i=1}^n u_i \phi_i)}{\partial t} \phi_I - \nu \int_{\Omega} \nabla \left(\sum_{i=1}^n u_i \phi_i \right) \cdot \nabla \phi_I \\ + \int_{\Omega} \nabla \left(\sum_{j=1}^m p_j \psi_j \right) \cdot \phi_I &= \int_{\Omega} f \phi_I, \\ - \int_{\Omega} \left(\nabla \cdot \left(\sum_{i=1}^n u_i \phi_i \right) \right) \psi_J &= 0. \end{aligned} \quad (44)$$

After rearranging scalars, using properties of the dot product and the operator ∇ , and noting that ϕ_i does not depend on t , (44) can be formulated as (45).

$$\begin{aligned} \int_{\Omega} \left(\sum_{i=1}^n \phi_i \frac{\partial u_i}{\partial t} \right) \phi_I - \nu \int_{\Omega} \left(\sum_{i=1}^n u_i \nabla \phi_i \right) \cdot \nabla \phi_I \\ + \int_{\Omega} \left(\sum_{j=1}^m p_j \nabla \psi_j \right) \cdot \phi_I &= \int_{\Omega} f \phi_I, \quad I = 1, \dots, n \\ - \int_{\Omega} \left(\sum_{i=1}^n u_i \nabla \cdot \phi_i \right) \psi_J &= 0, \quad J = 1, \dots, m. \end{aligned} \quad (45)$$

Finally, (45) can be formulated as (46) after swapping integrals with summations and rearranging integration scalars.

$$\begin{aligned}
 & \sum_{i=1}^n \frac{\partial u_i}{\partial t} \int_{\Omega} \phi_i \phi_I - \sum_{i=1}^n u_i \int_{\Omega} \nu \nabla \phi_i \cdot \nabla \phi_I \\
 & + \sum_{j=1}^m p_j \int_{\Omega} \phi_I \cdot \nabla \psi_j = \int_{\Omega} f \phi_I, \quad I = 1, \dots, n \\
 & - \sum_{i=1}^n u_i \int_{\Omega} \psi_J \nabla \cdot \phi_i = 0, \quad J = 1, \dots, m.
 \end{aligned} \tag{46}$$

As before, the problem can be reduced to the matrix system (47), which is the semi-discrete Stokes problem [8].

$$\begin{aligned}
 M\dot{u} + Lu + Gp &= f, \\
 -Du &= 0,
 \end{aligned} \tag{47}$$

where M and L are $n \times n$ matrices, G is a $n \times m$ matrix, D is a $m \times n$ matrix, u and f are n -dimensional vectors, p is a m -dimensional vector and $\dot{u}$ is the notation used for the partial derivate of u with respect to time t . The entries of these arrays are $M[i, j] = \int_{\Omega} \phi_i \phi_j$, $L[i, j] = \int_{\Omega} \nu \nabla \phi_i \cdot \nabla \phi_j$, $G[i, j] = \int_{\Omega} \phi_i \cdot \nabla \psi_j$, $D[i, j] = \int_{\Omega} \psi_i \nabla \cdot \phi_j$, $f[i] = \int_{\Omega} f \phi_i$, $p[i] = p_i$, $u[i] = u_i$ and $\dot{u}[i] = \frac{\partial u_i}{\partial t}$.

As mentioned on [8], for the steady Stokes problem, $\dot{u} = 0$ is taken. In such case, (47) becomes the linear matrix system (48).

$$\begin{pmatrix} L & G \\ -D & 0 \end{pmatrix} \begin{pmatrix} u \\ p \end{pmatrix} = \begin{pmatrix} f \\ 0 \end{pmatrix} \tag{48}$$

Furthermore, for the Navier-Stokes equations (41), the semi-discrete formulation is [8]:

$$\begin{aligned}
 M\dot{u} + Lu + \mathcal{N}(u) + Gp &= f, \\
 -Du &= 0,
 \end{aligned} \tag{49}$$

where

$$\mathcal{N}(u)[i] = \int_{\Omega} \begin{pmatrix} u_1 & \dots & u_n \end{pmatrix} \begin{pmatrix} (\phi_1 \cdot \nabla) \phi_1 & \dots & (\phi_1 \cdot \nabla) \phi_n \\ \vdots & \ddots & \vdots \\ (\phi_n \cdot \nabla) \phi_1 & \dots & (\phi_n \cdot \nabla) \phi_n \end{pmatrix} \begin{pmatrix} u_1 \\ \vdots \\ u_n \end{pmatrix} \phi_i$$

is the discretized nonlinear vector-convection term, of size $n \times n$.

Finally, recall that the spaces U_h^k and P_h^k have a given order k . As mentioned on [8], Taylor-Hood finite element space is the tuple (U_h^k, P_{k-1}^k) , which

is used to solve steady and unsteady Stokes problem (convergence is optimal and stable for $k \geq 2$). And, for Navier-Stokes equations, the finite element space used is (U_h^k, P_h^k) , called $P_N P_N$ space.

According to MFEM documentation [5], the implementation for the solution of Navier-Stokes equations is done following [8], which is the theory presented on this section.

1.3. MFEM Library

In this manuscript, we worked with MFEM's *Example#1* and *Example#5* which can be found on [5]. *Example#1* uses standard Lagrange finite elements and *Example#5* uses Raviart-Thomas mixed finite elements. Further, in Section 2.1, we find the parameters so that both problems are equivalent and then (Section 2.4), we compare the solutions.

We finally mention that for a fair comparison between Lagrange and mixed finite element's approximation, Lagrange shape functions of order $k - 1$ will be compared to the corresponding (mixed) approximation obtained by using $RT_k(K)$.

Afterwards, we worked with MFEM's miniapp for solving Navier-Stokes equations, which corresponds to the experiments done on Section 3. On Section 3.1 we worked in a 2-dimensional domain, and on Section 3.2 we worked in a 3-dimensional domain.

1.3.1. Information about the library

According to its official site [5], MFEM is a free, lightweight, scalable C++ library for finite element methods that can work with arbitrary high-order finite element meshes and spaces.

MFEM has a serial and a parallel version. The serial version is the one recommended for beginners, and is used in Section 2. On the other hand, the parallel version provides more computational power and enables the use of some MFEM *mini-apps*, like the Navier-Stokes mini app, which is used in Section 3.

Moreover, the Modular Finite Element Method (MFEM) library is developed by the MFEM Team at the Center for Applied Scientific Computing (CASC), located in the Lawrence Livermore National Laboratory (LLNL), under the BSD licence. However, as it is open source, the public repository can be found at <https://github.com/mfem> in order for anyone to contribute.

Also, since 2018, a wrapper for Python (PyMFEM) is being developed in order to use MFEM library among with Python code, which demonstrates the wide

applicability that the library can achieve. And, in 2021, the first community workshop was hosted by the MFEM Team, which encourages the use of the library and enlarges the community of MFEM users.

Finally, take into account that the use of the library requires a good manage of C++ code, which is a programming language that's harder to use compared to other languages, such as Python. This understanding of C++ code is important because some parts of the library are not well documented yet, and, by checking the source code, the user may find a way of implementing what is required.

1.3.2. Overview

The main classes (with a brief and superficial explanation of them) that we are going to use in the code are:

- Mesh: domain with the partition.
- FiniteElementSpace: space of functions defined on the finite element mesh.
- GridFunction: mesh with values (solutions).
- _Coefficient: values of GridFunctions or constants.
- LinearForm: maps an input function to a vector for the rhs.
- BilinearForm: used to create a global sparse finite element matrix for the lhs.
- _Vector: vector.
- _Solver: algorithm for solution calculation.
- _Integrator: evaluates the bilinear form on element's level.
- NavierSolver: class associated to the navier mini-app, which is used to solve Navier-Stokes equations.

The ones that have `_` are various classes whose name ends up the same and work similarly.

Note:

lhs: left hand side of the linear system.

rhs: right hand side of the linear system.

1.3.3. Code structure

A MFEM general code has the following steps (directly related classes with the step are written):

1. Receive an input file (.msh) with the mesh and establish the order for the finite element spaces.
2. Create a mesh object, get the dimension, and refine the mesh (refinement is optional). **Mesh**
3. Define the finite element spaces required. **FiniteElementSpace**
4. Define the coefficients, functions, and boundary conditions of the problem. **XCoefficient**
5. Define the LinearForm for the rhs and assemble it. **LinearForm**, **XIntegrator**
6. Define the BilinearForm for the lhs and assemble it. **BilinearForm**, **XIntegrator**
7. Solve the linear system. **XSolver**, **XVector**
8. Recover solution. **GridFunction**
9. Show solution with a finite element visualization tool like **GLVis** [6] (optional).

And, for the general code structure of the navier mini-app, we have the following steps:

1. Receive an input file (.msh) with the mesh, create a parallel mesh object and refine the mesh (refinement is optional). **ParMesh**
2. Create the flow solver by stating the order of the finite element spaces and the parameter for kinematic viscosity ν . **NavierSolver**
3. Establish the initial condition, the boundary conditions and the time step dt .
4. Iterate through steps in time with the NavierSolver object and save the solution for each iteration in a parallel GridFunction. **ParGridFunction**
5. Show the solution with a finite element visualization tool like **ParaView** [10] (optional).

Notice that the Mesh and GridFunction classes used in the mini-app are for the parallel version of MFEM. The reason for this, is that the navier mini-app is available for the parallel version of MFEM only. Also, the mini-app is coded in such way that the code is simple (see Appendix 5.3).

2. Lagrange vs. Raviart-Thomas finite elements

In this section, we take examples 1 and 5 from [5], define their problem parameters in such way that they're equivalent, create a code that implements both of them at the same time and compares both solutions (L_2 norm), run the code with different orders, and analyse the results.

Some considerations to have into account for a fair comparison are that, the order for the Mixed method should be 1 less than the order for Lagrange method, because, with this, both shape functions would have the same degree. Also, we will compare pressures and velocities with respect to the order of the shape functions and the size of the mesh (h parameter). Furthermore, for the problem, the exact solution is known, so, we will use it for comparison. And, the maximum order and refinement level to be tested is determined by our computational capacity (as long as solvers converge fast).

2.1. Problem

As mentioned before, we have to find the parameters for example 1 and 5 from [5], in such way that both problems are equivalent. This step is important because example # 1 is solved using Lagrange finite elements, while example # 5 is solved using mixed finite elements. Therefore, in order to make the comparison, the problem must be the same for both methods.

Example#1 [5]: Compute p such that

$$\begin{aligned} -\Delta p &= 1 \text{ in } \Omega \\ p &= 0 \text{ in } \Gamma. \end{aligned} \tag{50}$$

Example#5 [5]: Compute p and $\mathbf{u}$ such that

$$\begin{aligned} k\mathbf{u} + \nabla p &= f \text{ in } \Omega \\ -\operatorname{div}(\mathbf{u}) &= g \text{ in } \Omega \\ -p &= p_0 \text{ in } \Gamma. \end{aligned} \tag{51}$$

From the first equation of (51),

$$\mathbf{u} = \frac{f - \nabla p}{k}. \tag{52}$$

Then, replacing (52) on the second equation of (51),

$$-\operatorname{div}\left(\frac{f - \nabla p}{k}\right) = g. \tag{53}$$

If we set $k = 1$; $f = 0$ and $g = -1$ in (53), we get

$$-\Delta p = 1, \tag{54}$$

which is the first equation of (50).

So, setting (*) $p_0 = 0$, $k = 1$; $f = 0$ and $g = -1$ in (51), we get,

$$\begin{aligned} \mathbf{u} + \nabla p &= 0 \text{ in } \Omega \\ -\operatorname{div}(\mathbf{u}) &= -1 \text{ in } \Omega \\ -p &= 0 \text{ in } \Gamma. \end{aligned} \tag{55}$$

Notice that from the first equation we get that $\mathbf{u} = -\nabla p$. This is important because in problem (50) we don't get the solution for $\mathbf{u}$ from the method, so, we will have to find it from p 's derivatives.

In the code, we will set the value of the parameters in the way shown here, so that both problems are the same. As seen in (52)-(54), problem (51) is equivalent to problem (50) with the values assigned for coefficients and functions at (*).

2.2. Code

The first part of the code follows the structure mentioned in Section 1.3.3, but implemented for two methods at the same time (and with some extra lines for comparison purposes). Also, when defining boundary conditions, the *essential* one is established different from the *natural* one. And, after getting all the solutions, there's a second part of the code where solutions are compared between them and with the exact one.

Note:

The complete code with explanations can be found on the **Appendix A**.

However, before taking a look into it, the reader may have the following into account. The following table shows the convention used for important variable names along the code:

Variable Name	Object
X_space	Finite element space X
X_mixed	Variable assigned to a mixed method related object
u	Velocity solution
p	Pressure solution
X_ex	Variable assigned to an exact solution object

2.3. Tests

The tests of the two methods presented previously, were run on MFEM library on the domain shown on Figure 6.

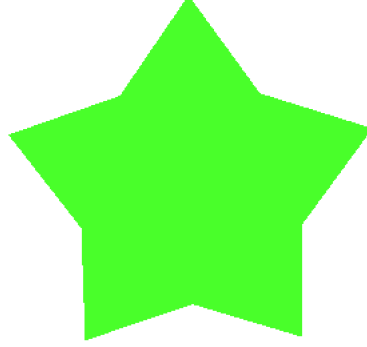


Figure 6: Illustration of the star domain used for the numerical tests.
Visualization: [6].

Each run test is determined by the *order* of the Lagrange shape functions and the h parameter of the mesh. Remember that mixed shape functions have order equal to $order - 1$. The parameter *order* is changed directly from the command line, while the parameter h is changed via the number of times that the mesh is refined ($h = h(\#refinements)$). As we refine the mesh more times, finite elements of the partition decrease their size, and so, the parameter h decreases.

Tests were run with: $order = 1, \dots, N$ and $refinements = 0, \dots, M$, where N, M depend on the computation capacity. The star domain was partitioned using quads (instead of triangles), and such partition is shown on Figure 7.

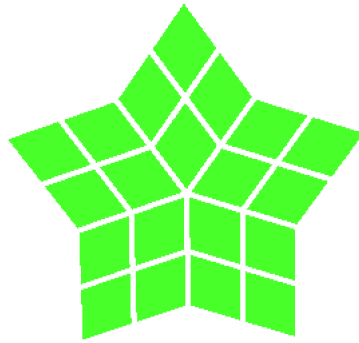


Figure 7: Initial mesh used for numerical tests (no refinements).
Visualization: [6].

Results on Section 2.4 are presented in graphs. However, all the exact values that were computed can be found in the **Appendix B**.

2.4. Results

In Figure 8 we show the computed solution when running the code with $order = 2$ and $\#Refinements = 3$. We use the visualization tool [6]. We mention that, at the scale of the plot, Lagrange and Mixed solutions look the same.

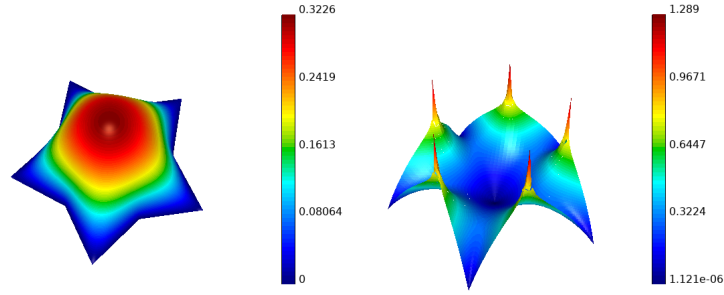


Figure 8: Illustration of computed pressure and velocities. GLVis ([6]) is used for this visualization. Pressure (left) and L^2 norm of the vector velocity (right). Visualization: [6].

In the following results, if $u = (u_x, u_y)$ is the solution obtained by the mixed or Lagrange finite element method and $u_{ex} = (u_{x_{ex}}, u_{y_{ex}})$ is the exact solution for the problem, then,

$$U_{error} = \frac{\sqrt{(\|u_x - u_{x_{ex}}\|_{L^2})^2 + (\|u_y - u_{y_{ex}}\|_{L^2})^2}}{\|u_{ex}\|_{L^2}}.$$

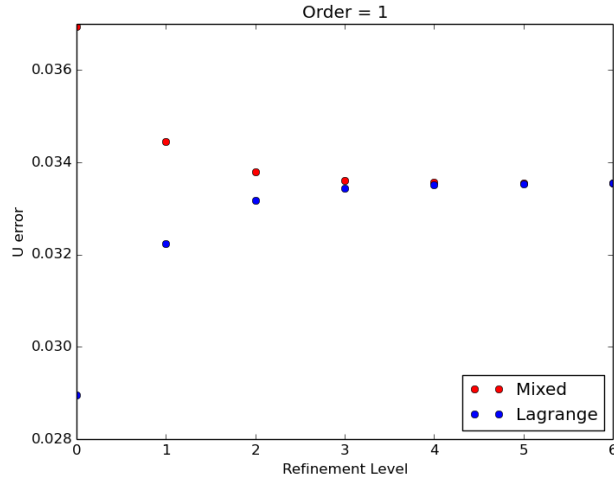


Figure 9: Variation of error with respect to the refinement level for the approximation of the solution of problem (50) with Lagrangian finite elements of order 1 and problem (51) with mixed Raviart-Thomas finite elements of order 0.

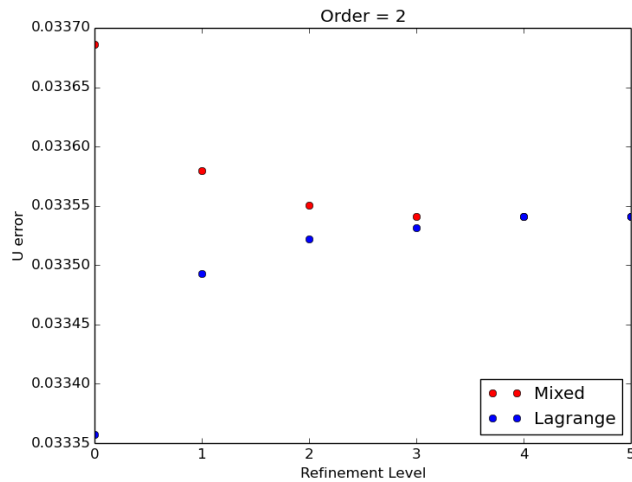


Figure 10: Variation of error with respect to the refinement level for the approximation of the solution of problem (50) with Lagrangian finite elements of order 2 and problem (51) with mixed Raviart-Thomas finite elements of order 1.

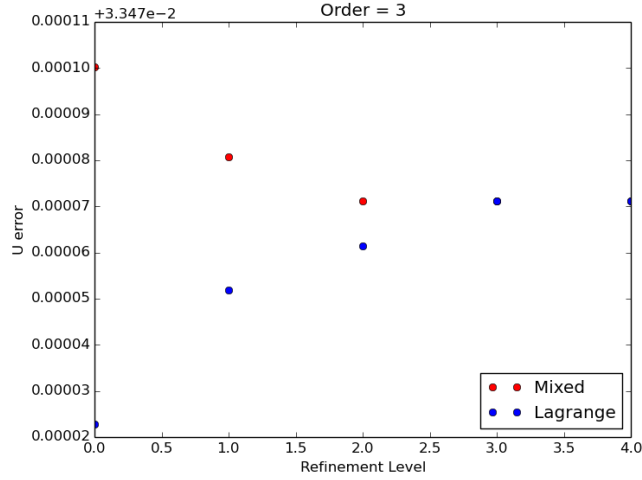


Figure 11: Variation of error with respect to the refinement level for the approximation of the solution of problem (50) with Lagrangian finite elements of order 3 and problem (51) with mixed Raviart-Thomas finite elements of order 2.

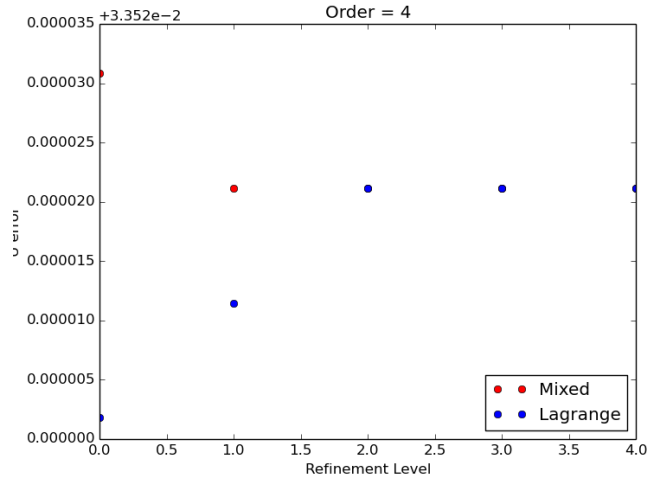


Figure 12: Variation of error with respect to the refinement level for the approximation of the solution of problem (50) with Lagrangian finite elements of order 4 and problem (51) with mixed Raviart-Thomas finite elements of order 3.

2.5. Analysis

In this section we comment and analyze the results in tables presented on the Appendix B.

To understand the information presented, take into account that the exact solution would have value 0 in *X err*. Also, if the two solutions obtained (Lagrange and Mixed) are exactly the same, the value in *P comp* and *U comp* would be 0. And, lower values of h mean more mesh refinements, ie, smaller partition elements.

As expected, computational time increases as order and refinements increase. Here are the most relevant observations that can be obtained after analysing the data corresponding to **absolute errors**.

For fixed order, absolute errors have little variation when reducing h (max variation is $4.722\text{e}-03$ in *U err* order 1); *P err* increases as h decreases, while *P mx err* decreases as h decreases; and, *U err* increases as h decreases, while *U mx err* decreases as h decreases.

Absolute errors variation (respect to refinement) is lower when order is higher. For example; in order 2, *P err* is the same for each h (up to three decimal places); while in order 6, *P err* is the same for each h (up to five decimal places).

For fixed h , absolute errors remain almost constant between orders. Moreover, *P err* (absolute error obtained for pressure with Lagrange) is always lower than *P mx err* (absolute error obtained for pressure with mixed) and *U err* (absolute error obtained for velocity with Lagrange) is always lower than *U mx err* (absolute error obtained for velocity with mixed).

As order increases, pressure and velocity absolute errors tend to be the same. In order 10, the difference between *P err* and *P mx err* is 0.000001 and the difference between *U err* and *U mx err* is < 0.0000009 .

However, notice that in all the cases, the absolute error was higher than 1. In L_2 norm, this value is pretty little, and shows that we are only getting approximations of the exact solution.

Now, the most relevant observations that can be obtained after analysing the data corresponding to **comparison errors**. First of all, comparison error tends to 0; and comparison errors *U comp* and *P comp* decrease as h decreases.

For a fixed order, comparison error can be similar to a higher order comparison error, as long as enough refinements are made. Moreover, when order increases, comparison errors are lower for fixed h .

Pressure comparison error lowers faster than velocity comparison error. Maximum comparison errors were found at order 1 with no refinements, where $Pcomp \approx 7.5e-02$ and $Ucomp \approx 3.7e-02$, and minimum comparison errors were found at order 10 with 1 refinement (higher refinement level computed for order 10), where $Pcomp \approx 5.1e-06$ and $Ucomp \approx 9.8e-04$. It can be seen that $Pcomp$ improved in almost four decimal places while $Ucomp$ improved in just 2.

2.6. Some other examples

In this section we show three of the examples that MFEM library provides [5]. We only show the problem, a brief verification of the exact solution and the solution obtained using MFEM, without going into details of any type. The purpose is to show the wide variety of applications that MFEM library can have and let the reader familiarize with the visualization of some finite element method solutions.

Example 1

This example is Example # 3 of [5] and consists on solving the second order definite Maxwell equation

$$\nabla \times \nabla \times E + E = f$$

with Dirichlet boundary condition. In the example, the value for f is given by

$$f \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} (1 + \pi^2) \sin(\pi y) \\ (1 + \pi^2) \sin(\pi z) \\ (1 + \pi^2) \sin(\pi x) \end{pmatrix}.$$

The exact solution for E is

$$E \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} \sin(\pi y) \\ \sin(\pi z) \\ \sin(\pi x) \end{pmatrix},$$

and can be verified by computing

$$\begin{aligned} & \nabla \times \nabla \times E + E \\ &= \begin{pmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{pmatrix} \times \begin{pmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{pmatrix} \times \begin{pmatrix} \sin(\pi x) \\ \sin(\pi z) \\ \sin(\pi y) \end{pmatrix} + \begin{pmatrix} \sin(\pi y) \\ \sin(\pi z) \\ \sin(\pi x) \end{pmatrix} \\ &= \begin{pmatrix} \frac{\partial}{\partial x} \\ \frac{\partial}{\partial y} \\ \frac{\partial}{\partial z} \end{pmatrix} \times \begin{pmatrix} -\pi \cos(\pi z) \\ -\pi \cos(\pi x) \\ -\pi \cos(\pi y) \end{pmatrix} + \begin{pmatrix} \sin(\pi y) \\ \sin(\pi z) \\ \sin(\pi x) \end{pmatrix} \\ &= \begin{pmatrix} \pi^2 \sin(\pi y) \\ \pi^2 \sin(\pi z) \\ \pi^2 \sin(\pi x) \end{pmatrix} + \begin{pmatrix} \sin(\pi y) \\ \sin(\pi z) \\ \sin(\pi x) \end{pmatrix} = f. \end{aligned}$$

The solution for E , E_h , computed with MFEM library is presented on Figure 13. The error for the approximation is

$$\|E_h - E\|_{L^2} = 0.39154.$$

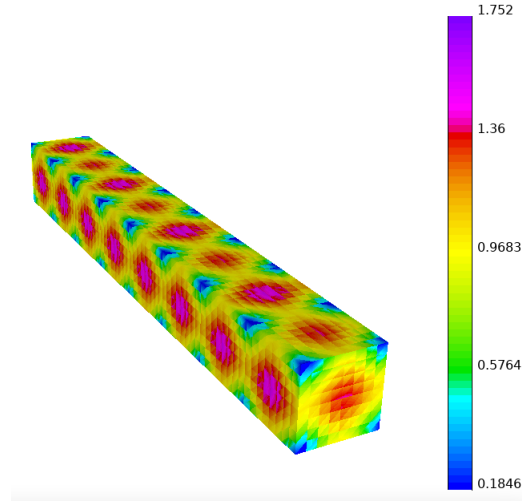


Figure 13: Visualization of the norm of the solution for the electromagnetic diffusion problem corresponding to the second order definite Maxwell equation. Solution for E , E_h , obtained using MFEM library. Visualization: [6].

Example 2

This example is Example # 4 of [5] and consists on solving the diffusion problem corresponding to the second order definite equation

$$-\nabla(\alpha \operatorname{div}(F)) + \beta F = f$$

with Dirichlet boundary condition, and, with parameters $\alpha = 1$ and $\beta = 3$. In the example, the value for f is given by

$$f \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} (3 + 2\pi^2) \cos(\pi x) \sin(\pi y) \\ (3 + 2\pi^2) \cos(\pi y) \sin(\pi x) \end{pmatrix}.$$

The exact solution for F is

$$F \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} \cos(\pi x) \sin(\pi y) \\ \cos(\pi y) \sin(\pi x) \end{pmatrix},$$

and can be verified by computing

$$\begin{aligned}
& -\nabla(\alpha \operatorname{div}(F)) + \beta F \\
&= -\nabla \left(\operatorname{div} \begin{pmatrix} \cos(\pi x) \sin(\pi y) \\ \cos(\pi y) \sin(\pi x) \end{pmatrix} \right) + 3 \begin{pmatrix} \cos(\pi x) \sin(\pi y) \\ \cos(\pi y) \sin(\pi x) \end{pmatrix} \\
&= -\nabla \left(-2\pi \sin(\pi x) \sin(\pi y) \right) + 3 \begin{pmatrix} \cos(\pi x) \sin(\pi y) \\ \cos(\pi y) \sin(\pi x) \end{pmatrix} \\
&= - \begin{pmatrix} -2\pi^2 \cos(\pi x) \sin(\pi y) \\ -2\pi^2 \sin(\pi x) \cos(\pi y) \end{pmatrix} + 3 \begin{pmatrix} \cos(\pi x) \sin(\pi y) \\ \cos(\pi y) \sin(\pi x) \end{pmatrix} \\
&= \begin{pmatrix} 2\pi^2 \cos(\pi x) \sin(\pi y) \\ 2\pi^2 \cos(\pi y) \sin(\pi x) \end{pmatrix} + \begin{pmatrix} 3 \cos(\pi x) \sin(\pi y) \\ 3 \cos(\pi y) \sin(\pi x) \end{pmatrix} = f.
\end{aligned}$$

The solution for F , F_h , computed with MFEM library is presented on Figure 14. The domain is a square with a circular hole in the middle. On the visualization, the triangular elements of the mesh are shown. The error for the approximation is

$$\|F_h - F\|_{L^2} = 5.55372 \times 10^{-6}.$$

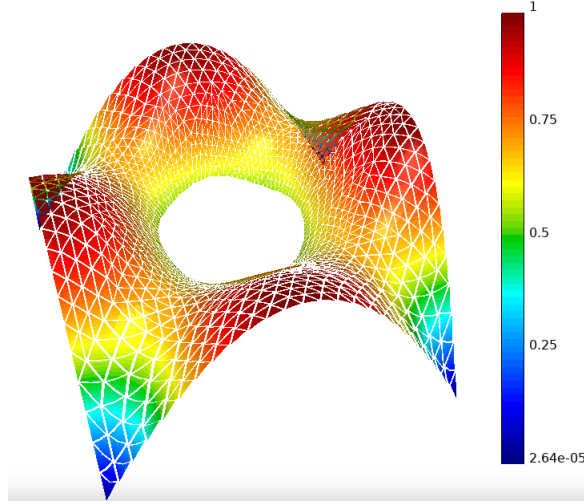


Figure 14: Visualization of the norm of the solution for the diffusion problem showing the mesh elements. Solution for F , F_h , obtained using MFEM library. Visualization: [6].

Example 3

This example is Example # 7 of [5] and consists on solving the Laplace problem with mass term corresponding to the equation

$$-\Delta u + u = f.$$

In the example, the value for f is given by

$$f \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \frac{7xy}{x^2 + y^2 + z^2}.$$

The exact solution for u is

$$u \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \frac{xy}{x^2 + y^2 + z^2},$$

and can be verified by computing

$$\begin{aligned} & -\Delta u + u \\ &= -\operatorname{div} \left(\nabla \left(\frac{xy}{x^2 + y^2 + z^2} \right) \right) + \frac{xy}{x^2 + y^2 + z^2} \\ &= -\operatorname{div} \left(\frac{1}{(x^2 + y^2 + z^2)^2} \begin{pmatrix} y(-x^2 + y^2 + z^2) \\ x(x^2 - y^2 + z^2) \\ -2xyz \end{pmatrix} \right) + \frac{xy}{x^2 + y^2 + z^2} \\ &= - \left(-\frac{6xy}{x^2 + y^2 + z^2} \right) + \frac{xy}{x^2 + y^2 + z^2} \\ &= \frac{7xy}{x^2 + y^2 + z^2} = f. \end{aligned}$$

The solution for u , u_h , computed with MFEM library is presented on Figure 15. The error for the approximation is

$$\|u_h - u\|_{L^2} = 0.00236119.$$

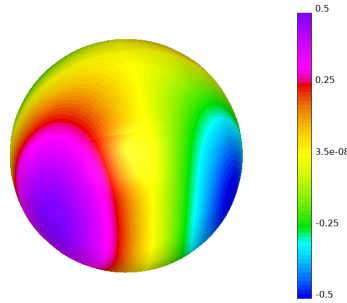


Figure 15: Visualization of the solution for the Laplace problem with mass term. Solution for u , u_h , obtained using MFEM library. Visualization: [6].

These examples show that MFEM can solve several types of equations that can include divergence, curl, gradient and Laplacian operators. Also, the given parameter f was a function of the form $\mathbb{R}^3 \rightarrow \mathbb{R}^3$, $\mathbb{R}^2 \rightarrow \mathbb{R}^2$ and $\mathbb{R}^3 \rightarrow \mathbb{R}$, which shows that MFEM can work with scalar and vectorial functions in a 2D or 3D domain. Although the shown visualizations were norms of the solution, using another visualization tool, such as [10], the vectors of the solution can be seen.

3. Numerical Experiments with NS

In this section we run some computational experiments solving Navier-Stokes equations using MFEM's Navier Stokes mini app. One of the experiments is in a 2D domain (Section 3.1) and the other one is in a 3D domain (Section 3.2). The two dimensional experiment converges to a steady state when the initial condition is the steady state with a small perturbation. In such experiment, we compare graphically the pressure solution obtained when changing the order and the refinement level. On the other hand, in the three dimensional experiment, we revise a graphical solution for a dynamical system, where turbulence is present.

3.1. 2D Experiment: Steady State

The two dimensional domain is a 2×4 rectangle whose vertex coordinates are $(-0.5, 1.5)$, $(-0.5, -0.5)$, $(1, 1.5)$ and $(1, -0.5)$, as shown on Figure 16.

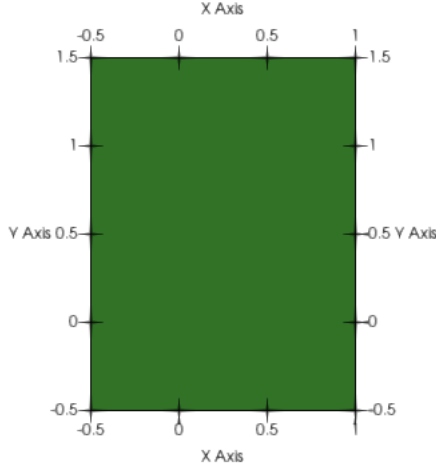


Figure 16: Two dimensional domain used in the 2D Navier-Stokes experiment. It is a 2×4 rectangle. Visualization: [10].

Also, the default mesh (with no refinements) is presented on figure 17.

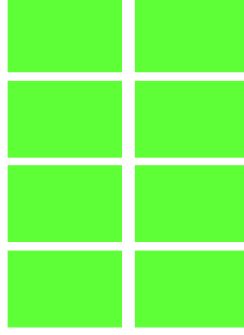


Figure 17: Two dimensional domain mesh used in the 2D Navier-Stokes experiment. Visualization: [10].

For this experiment, $Re = 40$ and the velocity boundary condition was settled to be

$$u_0(x, y) = \left(\begin{array}{c} 1 - e^{2(10 - \sqrt{100 + \pi^2})x} \cdot \cos(2\pi y) \\ \frac{2(10 - \sqrt{100 + \pi^2})}{2\pi} \cdot e^{2(10 - \sqrt{100 + \pi^2})x} \cdot \sin(2\pi y) \end{array} \right). \quad (56)$$

If the initial condition is picked to be u_0 , then the system is already on a steady state. As we wanted the experiment to *reach* the steady state, then initial condition was ensured to be

$$u_i(x, y) = u_0(x, y) + \delta \left(\begin{array}{c} (x + 0.5)(x - 1)(y + 0.5)(y - 1.5) \\ (x + 0.5)(x - 1)(y + 0.5)(y - 1.5) \end{array} \right), \quad (57)$$

where the δ parameter in the initial condition gives the magnitude of the perturbation from the steady state. The parameter was picked to be $\delta = 0.001$. Notice that the term after δ vanishes in the boundary of the domain.

The expected result in pressure (computed with order 6 and 5 refinements) is presented on figure 18, which corresponds to the steady state reached by the system.

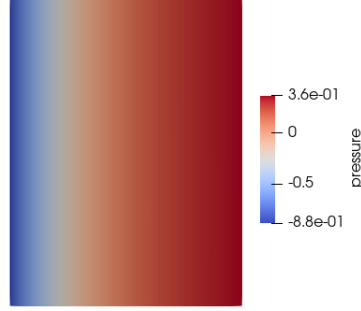


Figure 18: Expected solution for the pressure in the 2D experiment, computed with order 6 and 5 refinements. It is the steady state of the system. Visualization: [10].

For illustration of how the system changed, the initial condition for the pressure is shown in figure 19.

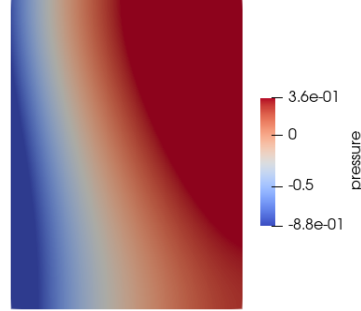


Figure 19: Initial condition for pressure in the 2D experiment, computed with order 6 and 5 refinements. Visualization: [10].

Notice that the initial condition has a higher pressure on the upper-right part of the domain and it's not uniform along any of the axes. However, after the system reaches the steady state pressure is constant for a fixed x value.

Now, the experiment consists on iterating through different orders and changing the refinement level for each of the orders, in order to check differences in the graphical solution. The experiment was done using a time step of $dt = 0.001$ with a total time of $T = 0.05$. It was computed with the parallel version of MFEM library (with 4 cores), using the Navier Miniapp [5]. The system was solved 36 times, corresponding to $order = 1, 2, 3, 4, 5, 6$, and for each of them, $\#refinements = 0, 1, 2, 3, 4, 5$. After checking all of the results in ParaView [10], we summarized the general behaviour of the steady state solutions, as

shown on figure 20. As notation, each of the results has a corresponding (k, r) value, where k denotes the order and r the amount of refinements made.

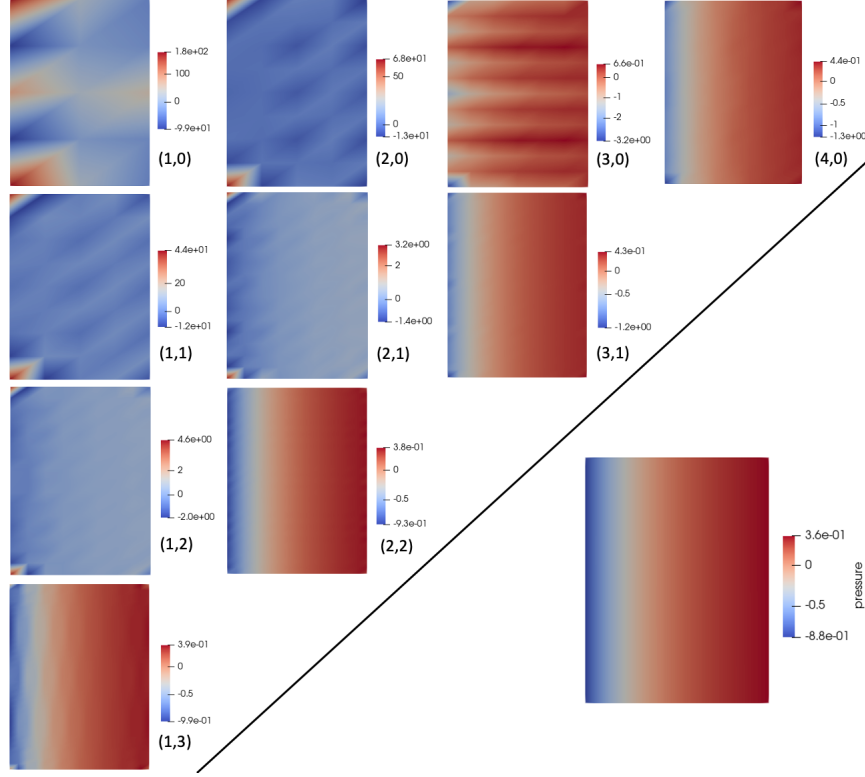


Figure 20: Summary of the steady state solutions for the 2D experiment as order and refinement level changes. The orders and refinement levels that don't appear are almost identical to the expected steady state. Visualization: [10].

From the summary it is clear that with low order and low refinement levels, the approximation for the solution is not a good one. It's interesting to notice that increasing 1 in the order can be similar to increasing 1 in the number of refinements, as seen on the steady state of $(1, 1) \sim (2, 0)$, $(1, 2) \sim (2, 1)$ and $(3, 1) \sim (4, 0)$.

Furthermore, with a high order, the solution converges as expected even if the mesh is not refined. Finally, note that for the lowest order or for the lowest refinement level, an increase of 5 in the counterpart achieves a good approximation (the steady states that are not shown are the ones that already achieve a good approximation).

3.2. 3D Experiment: Turbulence

The three dimensional domain is a $0.4 \times 0.4 \times 2.5$ parallelepiped with a vertex on the origin $(0, 0, 0)$ and the other ones on the positive part of the coordinate system. It has a cylindrical hole, parallel to the yx -plane, whose cross section center is located at $(0.5, 0.2)$ and has a radius of 0.05. The domain is shown on Figure 21.

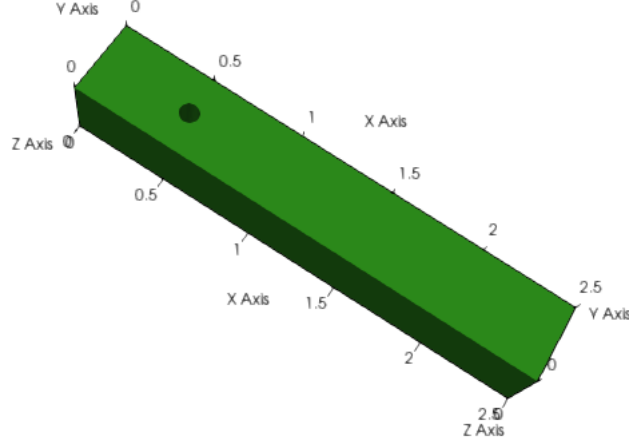


Figure 21: Three dimensional domain used in the 3D Navier-Stokes experiments. It is a $0.4 \times 0.4 \times 2.5$ parallelepiped with a cylindrical hole. Visualization: [10].

For the experiment, both the initial condition and the boundary condition for velocity were settled to be the same:

$$u_0 \begin{pmatrix} x \\ y \\ z \\ t \end{pmatrix} = u_i \begin{pmatrix} x \\ y \\ z \\ t \end{pmatrix} = \begin{pmatrix} u_x(x, y, z, t) \\ 0 \\ 0, \end{pmatrix} \quad (58)$$

where

$$u_x(x, y, z, t) = \begin{cases} \frac{36yz}{0.41^4} \sin\left(\frac{\pi t}{8}\right) (0.41 - y)(0.41 - z), & \text{if } x \leq 10^{-8}, \\ 0, & \text{if } x > 10^{-8}. \end{cases} \quad (59)$$

Notice that the velocity condition simulates a system where the fluid is entering through the squared face of the domain near the hole. Also, the boundary condition was forced only on the inlet and on the walls of the domain.

Furthermore, the experiment was computed with 4-th order elements, using a time step of $dt = 0.001$, a total time of $T = 8$ and the parameter of kinematic viscosity for the fluid being $\nu = 0.001$.

For the visualization of the solutions, we used ParaView's [10] stream tracer functionality, which shows the stream lines of the system at a given moment. Recall that the stream lines are tangent to the vector field of the velocity, and they show the trajectory of particles through the field (at a given instant of time). Also, there are 5 visualizations, corresponding to times $t = 0, 2, 4, 6, 8$, and have the value of pressure codified in a color scale.

At $t = 0$, the stream lines show a laminar flow (all lines are almost parallel and follow the same direction) that avoids the obstacle, and the pressure is high on the left because the fluid is entering through that part of the domain, as shown on figure 22.

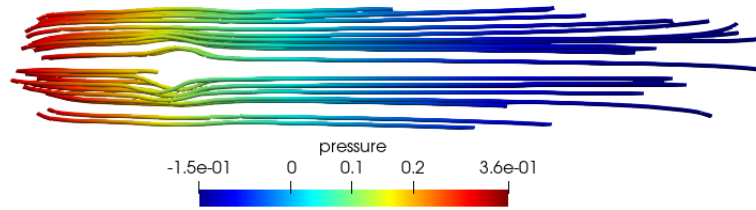


Figure 22: Stream lines for the velocity of the 3D experiment at $t = 0$. Visualization: [10].

Then, at $t = 2$, more fluid is coming in, therefore, the pressure on the left increases. However, the value of pressure after the obstacle starts to have some variations, which will cause the turbulence later.

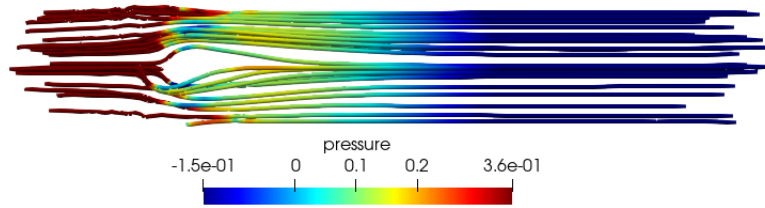


Figure 23: Stream lines for the velocity of the 3D experiment at $t = 2$. Visualization: [10].

At $t = 4$, the turbulence starts to show up. As seen on figure 24, after the obstacle, some of the stream lines have spiral forms.

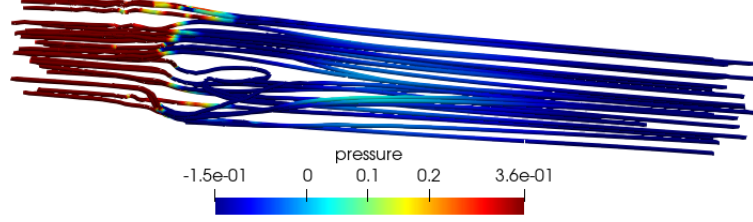


Figure 24: Stream lines for the velocity of the 3D experiment at $t = 4$. Visualization: [10].

Then, at $t = 6$, the pressure on the left finally starts to lower (with a lot of variation) and the pressure on the right starts to increase, because the fluid is already passing through the obstacle and no more fluid is coming in.. More spiral-shaped stream lines show up after the obstacle.

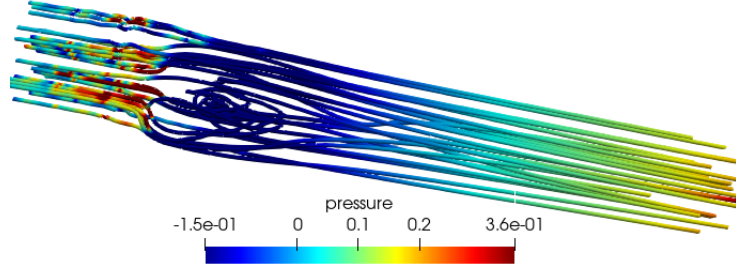


Figure 25: Stream lines for the velocity of the 3D experiment at $t = 6$. Visualization: [10].

Finally, at $t = T = 8$, the pressure on the left is low and on the right is high. However, the high variation of pressure in previous time steps generated a lot of turbulence near the obstacle. The fluid presents two states, a laminar one, and a turbulent one. The turbulent state, characterized by spiral movement and swirls, is presented near the obstacle; while the laminar state, characterized by straight lines, is presented away from the obstacle. As seen on figure 26.

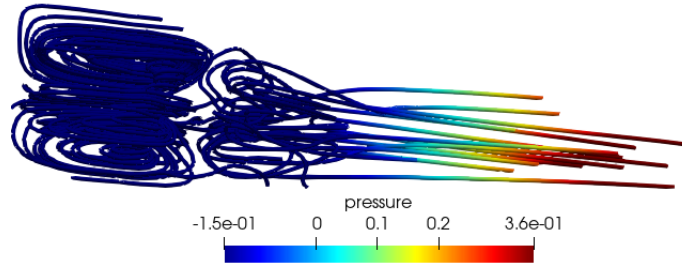


Figure 26: Stream lines for the velocity of the 3D experiment at $t = T = 8$. Visualization: [10].

4. Conclusion and Perspectives

The MFEM library allows us to approximate the solution of partial differential equations in a versatile way. Moreover, the library has a lot of potential because it can compute with high order elements without requiring a very powerful computer, for example, we could run experiments with elements of order 10 (which is a relatively high order). Also, the navier mini app of the library provides a simple, precise and efficient way for simulating dynamical systems that involve incompressible fluids. Furthermore, it is important to use a good visualization tool, preferably one that allows the visualization of vector fields and stream lines when working with fluid equations. Finally, recall that finite element methods enable the study of systems that depend on difficult partial differential equations.

Following this work, some study can be made on some of the following topics:

- Fluid modeling via partial differential equations.
- Error theorems for finite element methods.
- Effects of the mesh in the solution.
- Solutions for the equations when the parameters are not constants, but functions.
- Discretization of time.
- Picking of the time step dt , in order to achieve an appropriate solution.
- Turbulence and laminar flow.

References

- [1] Felipe Cruz. Comparing Lagrange and Mixed finite element methods using MFEM library. Beyond Research work at National University of Colombia. Arxiv: <https://arxiv.org/submit/3724279/view>
- [2] Claes Johnson. *Numerical Solution of Partial Differential Equations by the Finite Element Method*. ISBN10 048646900X. Dover Publications Inc. 2009.
- [3] Gabriel N. Gatica. *A Simple Introduction to the Mixed Finite Element Method*. Theory and Applications. ISBN 978-3-319-03694-6. Springer. 2014.
- [4] Juan Galvis & Henrique Versieux. *Introdução à Aproximação Numérica de Equações Diferenciais Parciais Via o Método de Elementos Finitos*. ISBN: 978-85-244-325-5. 28 Colóquio Brasileiro de Matemática. 2011.
- [5] R. Anderson and J. Andrej and A. Barker and J. Bramwell and J.-S. Camier and J. Cervený V. Dobrev and Y. Dudouit and A. Fisher and Tz. Kolev and W. Pazner and M. Stowell and V. Tomov and I. Akkerman and J. Dahm and D. Medina and S. Zampini, *MFEM: A modular finite element methods library*, *Computers & Mathematics with Applications* **81** (2021), 42-74. Main Page: <https://mfem.org>. DOI <https://doi.org/10.11578/dc.20171025.1248>
- [6] GLVis - OpenGL Finite Element Visualization Tool. Main Page: <https://glvis.org>. DOI <https://doi.org/10.11578/dc.20171025.1249>
- [7] Kalita, P. & Łukaszewicz, G. (2016). *Navier-Stokes Equations. An Introduction with Applications*. Springer. DOI <https://doi.org/10.1007/978-3-319-27760-8>.
- [8] Michael Franco, Jean-Sylvain Camier, Julian Andrej, Will Pazner (2020) High-order matrix-free incompressible flow solvers with GPU acceleration and low-order refined preconditioners (<https://arxiv.org/abs/1910.03032>)
- [9] Schneider, T., et al. (2018). *Decoupling Simulation Accuracy from Mesh Quality*. New York University, USA. 2018 Association for Computing Machinery. <https://doi.org/10.1145/3272127.3275067>.
- [10] Ayachit, Utkarsh, *The ParaView Guide: A Parallel Visualization Application*, Kitware, 2015, ISBN 978-1930934306.

5. Appendices

5.1. Appendix A : Code for comparison

Here, the code used for Section 2 (written in C++) is shown, with a brief explanations of it's functionality.

- ▷ Include the required libraries (including MFEM) and begin main function.

```
#include "mfem.hpp"
#include <fstream>
#include <iostream>
using namespace std;
using namespace mfem;
int main(int argc, char *argv[]) {
```

- ▷ Parse command-line options (in this project we only change "order" option) and print them.

```
const char *mesh_file = "../data/star.mesh";
int order = 1;
bool visualization = true;
OptionsParser args(argc, argv);
args.AddOption(&mesh_file, "-m", "--mesh",
               "Mesh_file_to_use.");
args.AddOption(&order, "-o", "--order",
               "Finite_element_order_(polynomial_degree).")
;
args.AddOption(&visualization, "-vis", "--visualization",
               , "-no-vis", "--no-visualization",
               "Enable_or_disable_GLVis_visualization.")
;
args.Parse();
if (!args.Good()) {
    args.PrintUsage(cout);
    return 1;
}
args.PrintOptions(cout);
```

- ▷ Create mesh object from the `star.mesh` file and get it's dimension.

```
Mesh *mesh = new Mesh(mesh_file, 1, 1);
int dim = mesh->Dimension();
```

▷ Refine the mesh a given number of times (**UniformRefinement**).

```
int ref_levels;
cout << "Refinements:␣";
cin >> ref_levels;
for (int l = 0; l < ref_levels; l++){
mesh->UniformRefinement();
}
```

▷ Get size indicator for mesh size (**h_max**) and print it.

```
double mesh_size, h = 0;
for (int i=0;i<mesh->GetNE();i++){
    mesh_size = mesh->GetElementSize(i,2);
    if(mesh_size>h){
        h = mesh_size;
    }
}
cout << "h:␣" << h << endl;
```

▷ Define finite element spaces. For mixed finite element method, the order will be one less than for Lagrange finite element method. The last one is a vector L^2 space that we will use later to get mixed velocity components.

```
FiniteElementCollection *H1 = new H1_FECollection(order,
    dim);
FiniteElementSpace *H1_space = new FiniteElementSpace(
    mesh, H1);
FiniteElementCollection *hdiv_coll(new RT_FECollection(
    order-1, dim));
FiniteElementCollection *l2_coll(new L2_FECollection(
    order-1, dim));
FiniteElementSpace *R_space = new FiniteElementSpace(
    mesh, hdiv_coll);
FiniteElementSpace *W_space = new FiniteElementSpace(
    mesh, l2_coll);
FiniteElementSpace *V_space = new FiniteElementSpace(
    mesh, l2_coll, 2);
```

- ▷ Define the parameters of the mixed problem. `C` functions are defined at the end. Boundary condition is natural.

```
ConstantCoefficient k(1.0);
void fFun(const Vector & x, Vector & f);
VectorFunctionCoefficient fcoeff(dim, fFun);
double gFun(const Vector & x);
FunctionCoefficient gcoeff(gFun);
double f_bound(const Vector & x);
FunctionCoefficient fbndcoeff(f_bound);
```

- ▷ Define the parameters of the Lagrange problem. Boundary condition is essential.

```
ConstantCoefficient one(1.0);
Array<int> ess_tdof_list;
if (mesh->bdr_attributes.Size()){
    Array<int> ess_bdr(mesh->bdr_attributes.Max());
    ess_bdr = 1;
    H1_space->GetEssentialTrueDofs(ess_bdr, ess_tdof_list);
}
```

- ▷ Define the exact solution. `C` functions are defined at the end.

```
void u_ex(const Vector & x, Vector & u);
double p_ex(const Vector & x);
double u_ex_x(const Vector & x);
double u_ex_y(const Vector & x);
```

- ▷ Get space dimensions and create vectors for the right hand side.

```
Array<int> block_offsets(3);
block_offsets[0] = 0;
block_offsets[1] = R_space->GetVSize();
block_offsets[2] = W_space->GetVSize();
block_offsets.PartialSum();
BlockVector rhs_mixed(block_offsets);
Vector rhs(H1_space->GetVSize());
```

- ▷ Define the right hand side. These are **LinearForm** objects associated to some finite element space and rhs vector. "f" and "g" are for the mixed method and "b" is for the Lagrange method. Also, "rhs" vectors are the variables that store the information of the right hand side.

```

LinearForm *fform(new LinearForm);
fform->Update(R_space, rhs_mixed.GetBlock(0), 0);
fform->AddDomainIntegrator(new
    VectorFEDomainLFIntegrator(fcoeff));
fform->AddBoundaryIntegrator(new
    VectorFEBoundaryFluxLFIntegrator(fbndcoeff));
fform->Assemble();

LinearForm *gform(new LinearForm);
gform->Update(W_space, rhs_mixed.GetBlock(1), 0);
gform->AddDomainIntegrator(new DomainLFIntegrator(gcoeff
));
gform->Assemble();

LinearForm *b(new LinearForm);
b->Update(H1_space, rhs, 0);
b->AddDomainIntegrator(new DomainLFIntegrator(one));
b->Assemble();

```

- ▷ Create variables to store the solution. "x" is the **Vector** used as input in the iterative method.

```

BlockVector x_mixed(block_offsets);
GridFunction u_mixed(R_space), p_mixed(W_space),
    ux_mixed(W_space), uy_mixed(W_space), ue(V_space);
Vector x(H1_space->GetVSize());
GridFunction ux(W_space), uy(W_space), p(H1_space);

```

- ▷ Define the left hand side for mixed method. This is the **BilinearForm** representing the Darcy matrix (39). **VectorFEMMassIntegrator** is associated to $k * u - \nabla p$ and **VectorFEDDivergenceIntegrator** is associated to $\text{div}(u)$.

```

BilinearForm *mVarf(new BilinearForm(R_space));
MixedBilinearForm *bVarf(new MixedBilinearForm(R_space,
    W_space));
mVarf->AddDomainIntegrator(new VectorFEMassIntegrator(k)
    );
mVarf->Assemble();
mVarf->Finalize();
SparseMatrix &M(mVarf->SpMat());
bVarf->AddDomainIntegrator(new
    VectorFEDivergenceIntegrator);
bVarf->Assemble();
bVarf->Finalize();
SparseMatrix &B(bVarf->SpMat());
B *= -1.;
SparseMatrix *BT = Transpose(B);
BlockMatrix D(block_offsets);
D.SetBlock(0,0, &M);
D.SetBlock(0,1, BT);
D.SetBlock(1,0, &B);

```

- ▷ Define the left hand side for Lagrange method. This is the **BilinearForm** associated to the laplacian operator. **DiffusionIntegrator** is associated to Δu . The method **FormLinearSystem** is only used to establish the essential boundary condition.

```

OperatorPtr A;
Vector XX, BB;
BilinearForm *a(new BilinearForm(H1_space));
a->AddDomainIntegrator(new DiffusionIntegrator(one));
a->Assemble();
a->FormLinearSystem(ess_tdof_list, p, *b, A, XX, BB);

```

- ▷ Solve linear systems with MINRES (for mixed) and CG (for Lagrange). **SetOperator** method establishes the lhs. **Mult** method executes the iterative algorithm and receives as input: the rhs and the vector to store the solution.

```

int maxIter(10000);
double rtol(1.e-6);
double atol(1.e-10);

MINRESSolver Msolver;
Msolver.SetAbsTol(atol);
Msolver.SetRelTol(rtol);
Msolver.SetMaxIter(maxIter);
Msolver.SetPrintLevel(0);
Msolver.SetOperator(D);
x_mixed = 0.0;
Msolver.Mult(rhs_mixed, x_mixed);
if (Msolver.GetConverged())
    std::cout << "MINRES_converged_in_" << Msolver.
        GetNumIterations() << "_iterations_with_a_residual_
        _norm_of_" << Msolver.GetFinalNorm() << ".\n";
else
    std::cout << "MINRES_did_not_converge_in_" << Msolver
        .GetNumIterations() << "_iterations._Residual_norm_
        _is_" << Msolver.GetFinalNorm() << ".\n";

CGSolver Lsolver;
Lsolver.SetAbsTol(atol);
Lsolver.SetRelTol(rtol);
Lsolver.SetMaxIter(maxIter);
Lsolver.SetPrintLevel(0);
Lsolver.SetOperator(*A);
x = 0.0;
Lsolver.Mult(rhs, x);
if (Lsolver.GetConverged())
    std::cout << "CG_converged_in_" << Lsolver.
        GetNumIterations() << "_iterations_with_a_residual_
        _norm_of_" << Lsolver.GetFinalNorm() << ".\n";
else
    std::cout << "CG_did_not_converge_in_" << Lsolver.
        GetNumIterations() << "_iterations._Residual_norm_
        _is_" << Lsolver.GetFinalNorm() << ".\n";

```

- ▷ Save the solution into **GridFunctions**, which are used for error computation and visualization.

```
u_mixed.MakeRef(R_space, x_mixed.GetBlock(0), 0);
p_mixed.MakeRef(W_space, x_mixed.GetBlock(1), 0);
p.MakeRef(H1_space, x, 0);
```

- ▷ Get missing velocities from the solutions obtained.
Remember that $u = -\nabla p$. Mixed components are extracted using the auxiliary variable "ue" defined before.

```
p.GetDerivative(1,0,ux);
p.GetDerivative(1,1,uy);
ux *= -1;
uy *= -1;

VectorGridFunctionCoefficient uc(&u_mixed);
ue.ProjectCoefficient(uc);
GridFunctionCoefficient ux_mixed_coeff(&ue,1);
GridFunctionCoefficient uy_mixed_coeff(&ue,2);
ux_mixed.ProjectCoefficient(ux_mixed_coeff);
uy_mixed.ProjectCoefficient(uy_mixed_coeff);
```

- ▷ Create the associated **Coefficient** objects for error computation.

```
GridFunction* pp = &p;
GridFunctionCoefficient p_coeff(pp);
GridFunction* uxp = &ux;
GridFunction* uyp = &uy;
GridFunctionCoefficient ux_coeff(uxp);
GridFunctionCoefficient uy_coeff(uyp);
FunctionCoefficient pex_coeff(p_ex);
VectorFunctionCoefficient uex_coeff(dim, u_ex);
FunctionCoefficient uex_x_coeff(u_ex_x);
FunctionCoefficient uex_y_coeff(u_ex_y);
```

- ▷ Define integration rule.

```
int order_quad = max(2, 2*order+1);
const IntegrationRule *irs [ Geometry::NumGeom ];
for (int i=0; i < Geometry::NumGeom; ++i) {
    irs[i] = &(IntRules.Get(i, order_quad));
}
```

- ▷ Compute exact solution norms. Here, the parameter 2 in `ComputeLpNorm` makes reference to the L^2 norm.

```
double norm_p = ComputeLpNorm(2., pex_coeff, *mesh, irs)
;
double norm_u = ComputeLpNorm(2., uex_coeff, *mesh, irs)
;
double norm_ux = ComputeLpNorm(2., uex_x_coeff, *mesh,
irs);
double norm_uy = ComputeLpNorm(2., uex_y_coeff, *mesh,
irs);
```

- ▷ Compute and print absolute errors.

```
double abs_err_u_mixed = u_mixed.ComputeL2Error(
uex_coeff, irs);
printf("Velocity_Mixed_Absolute_Error:_%e\n",
abs_err_u_mixed / norm_u);
double abs_err_p_mixed = p_mixed.ComputeL2Error(
pex_coeff, irs);
printf("Pressure_Mixed_Absolute_Error:_%e\n",
abs_err_p_mixed / norm_p);
double abs_err_p = p.ComputeL2Error(pex_coeff, irs);
printf("Pressure_Absolute_Error:_%e\n", abs_err_p /
norm_p);
double abs_err_ux = ux.ComputeL2Error(uex_x_coeff, irs);
double abs_err_uy = uy.ComputeL2Error(uex_y_coeff, irs);
double abs_err_u = pow(pow(abs_err_ux,2)+pow(abs_err_uy
,2),0.5);
printf("Velocity_Absolute_Error:_%e\n", abs_err_u /
norm_u);
```

- ▷ Compute and print comparison errors.

```
double err_ux = ux_mixed.ComputeL2Error(ux_coeff, irs);
double err_uy = uy_mixed.ComputeL2Error(uy_coeff, irs);
double err_u = pow(pow(err_ux,2)+pow(err_uy,2),0.5);
printf("Velocity_Comparison_Error:_%e\n", err_u / norm_u
);
double err_p = p_mixed.ComputeL2Error(p_coeff, irs);
printf("Pressure_Comparison_Error:_%e\n", err_p / norm_p
);
```

- ▷ Visualize the solutions and the domain. GLVis visualization tool uses port 19916 to receive data.

```

char vishost[] = "localhost";
int visport = 19916;
if(visualization){
    Vector x_domain(H1_space->GetVSize());
    GridFunction domain(H1_space);
    x_domain=0.0;
    domain.MakeRef(H1_space,x_domain,0);
    socketstream dom_sock(vishost, visport);
    dom_sock.precision(8);
    dom_sock << "solution\n" << *mesh << domain << "
        window_title_ 'Domain'" << endl;

    socketstream um_sock(vishost, visport);
    um_sock.precision(8);
    um_sock << "solution\n" << *mesh << u_mixed << "
        window_title_ 'Velocity_Mixed'" << endl;
    socketstream pm_sock(vishost, visport);
    pm_sock.precision(8);
    pm_sock << "solution\n" << *mesh << p_mixed << "
        window_title_ 'Pressure_Mixed'" << endl;
    socketstream uxm_sock(vishost, visport);
    uxm_sock.precision(8);
    uxm_sock << "solution\n" << *mesh << ux_mixed << "
        window_title_ 'X_Velocity_Mixed'" << endl;
    socketstream uym_sock(vishost, visport);
    uym_sock.precision(8);
    uym_sock << "solution\n" << *mesh << uy_mixed << "
        window_title_ 'Y_Velocity_Mixed'" << endl;

    socketstream p_sock(vishost, visport);
    p_sock.precision(8);
    p_sock << "solution\n" << *mesh << p << "
        window_title_ 'Pressure'" << endl;
    socketstream ux_sock(vishost, visport);
    ux_sock.precision(8);
    ux_sock << "solution\n" << *mesh << ux << "
        window_title_ 'X_Velocity'" << endl;
    socketstream uy_sock(vishost, visport);
    uy_sock.precision(8);
    uy_sock << "solution\n" << *mesh << uy << "
        window_title_ 'Y_Velocity'" << endl;
}
}

```

▷ Define $\mathbb{C}$ functions.

```

void fFun(const Vector & x, Vector & f){
    f = 0.0;
}
double gFun(const Vector & x){
    return -1.0;
}
double f_bound(const Vector & x){
    return 0.0;
}
void u_ex(const Vector & x, Vector & u){
    double xi(x(0));
    double yi(x(1));
    double zi(0.0);
    u(0) = - exp(xi)*sin(yi)*cos(zi);
    u(1) = - exp(xi)*cos(yi)*cos(zi);
}
double u_ex_x(const Vector & x){
    double xi(x(0));
    double yi(x(1));
    double zi(0.0);
    return -exp(xi)*sin(yi)*cos(zi);
}
double u_ex_y(const Vector & x){
    double xi(x(0));
    double yi(x(1));
    double zi(0.0);
    return -exp(xi)*cos(yi)*cos(zi);
}
double p_ex(const Vector & x){
    double xi(x(0));
    double yi(x(1));
    double zi(0.0);
    return exp(xi)*sin(yi)*cos(zi);
}

```

5.2. Appendix B : Numerical values of the comparison

The *order* parameter will be fixed for each table and *h* parameter is shown in the first column. To interpret the results take into account that **P** refers to pressure, **U** refers to velocity, **mx** refers to mixed (from mixed finite element method), **err** refers to absolute error (compared to the exact solution), and **comp** refers to comparison (the error between the two solutions obtained by the two different methods).

Order = 1

h	P comp	P err	Pmx err	U comp	U err	U mx err
0.572063	7.549479e-02	1.021287e+00	1.025477e+00	3.680827e-02	1.029378e+00	1.037635e+00
0.286032	3.627089e-02	1.022781e+00	1.023990e+00	1.727281e-02	1.032760e+00	1.035055e+00
0.143016	1.791509e-02	1.023236e+00	1.023596e+00	9.222996e-03	1.033725e+00	1.034369e+00
0.0715079	8.922939e-03	1.023372e+00	1.023480e+00	5.111295e-03	1.033999e+00	1.034182e+00
0.035754	4.455715e-03	1.023412e+00	1.023445e+00	2.859769e-03	1.034077e+00	1.034130e+00
0.017877	2.226845e-03	1.023424e+00	1.023435e+00	1.603788e-03	1.034100e+00	1.034115e+00

Order = 2

h	P comp	P err	Pmx err	U comp	U err	U mx err
0.572063	8.069013e-03	1.023329e+00	1.023554e+00	1.399079e-02	1.033924e+00	1.034255e+00
0.286032	2.138257e-03	1.023391e+00	1.023470e+00	7.845012e-03	1.034056e+00	1.034146e+00
0.143016	5.704347e-04	1.023417e+00	1.023442e+00	4.400448e-03	1.034093e+00	1.034120e+00
0.0715079	1.537926e-04	1.023426e+00	1.023434e+00	2.469526e-03	1.034104e+00	1.034112e+00
0.035754	4.194302e-05	1.023428e+00	1.023431e+00	1.385966e-03	1.034107e+00	1.034110e+00

Order = 3

h	P comp	P err	Pmx err	U comp	U err	U mx err
0.572063	8.691241e-04	1.023389e+00	1.023471e+00	8.745151e-03	1.034060e+00	1.034143e+00
0.286032	2.477673e-04	1.023417e+00	1.023443e+00	4.911967e-03	1.034094e+00	1.034120e+00
0.143016	7.316263e-05	1.023426e+00	1.023434e+00	2.756849e-03	1.034104e+00	1.034112e+00
0.0715079	2.178864e-05	1.023428e+00	1.023431e+00	1.547232e-03	1.034108e+00	1.034110e+00

Order = 4

h	P comp	P err	Pmx err	U comp	U err	U mx err
0.572063	3.199774e-04	1.023412e+00	1.023448e+00	6.119857e-03	1.034088e+00	1.034124e+00
0.286032	9.547574e-05	1.023424e+00	1.023435e+00	3.434952e-03	1.034103e+00	1.034114e+00
0.143016	2.862666e-05	1.023428e+00	1.023431e+00	1.927814e-03	1.034107e+00	1.034111e+00

Order = 5

h	P comp	P err	Pmx err	U comp	U err	U mx err
0.572063	1.552006e-04	1.023420e+00	1.023439e+00	4.578518e-03	1.034099e+00	1.034117e+00
0.286032	4.658038e-05	1.023427e+00	1.023433e+00	2.569749e-03	1.034106e+00	1.034112e+00
0.143016	1.406993e-05	1.023429e+00	1.023431e+00	1.442205e-03	1.034108e+00	1.034110e+00

Order = 6

h	P comp	P err	Pmx err	U comp	U err	U mx err
0.572063	8.612580e-05	1.023424e+00	1.023435e+00	3.584133e-03	1.034103e+00	1.034114e+00
0.286032	2.600417e-05	1.023428e+00	1.023431e+00	2.011608e-03	1.034107e+00	1.034111e+00
0.143016	7.897631e-06	1.023429e+00	1.023430e+00	1.128989e-03	1.034109e+00	1.034110e+00

Order = 7

h	P comp	P err	Pmx err	U comp	U err	U mx err
0.572063	5.243187e-05	1.023426e+00	1.023433e+00	2.899307e-03	1.034105e+00	1.034112e+00
0.286032	1.589631e-05	1.023429e+00	1.023431e+00	1.627221e-03	1.034108e+00	1.034110e+00

Order = 8

h	P comp	P err	Pmx err	U comp	U err	U mx err
0.572063	3.409225e-05	1.023427e+00	1.023432e+00	2.404311e-03	1.034107e+00	1.034111e+00
0.286032	1.037969e-05	1.023429e+00	1.023430e+00	1.349427e-03	1.034108e+00	1.034110e+00

Order = 9

h	P comp	P err	Pmx err	U comp	U err	U mx err
0.572063	2.328387e-05	1.023428e+00	1.023431e+00	2.033288e-03	1.034107e+00	1.034110e+00
0.286032	7.124397e-06	1.023429e+00	1.023430e+00	1.141177e-03	1.034109e+00	1.034110e+00

Order = 10

h	P comp	P err	Pmx err	U comp	U err	U mx err
0.572063	1.664200e-05	1.023429e+00	1.023431e+00	1.746755e-03	1.034108e+00	1.034110e+00
0.286032	5.085321e-06	1.023429e+00	1.023430e+00	9.803705e-04	1.034109e+00	1.034109e+00

5.3. Appendix C : MiniApp Code for Navier-Stokes

Here, the code used for Section 3 (written in C++) is shown, with a brief explanations of it's functionality.

- ▷ Include the required libraries (including navier miniapp).

```
#include "navier_solver.hpp"
#include <fstream>
using namespace mfem;
using namespace navier;
using namespace std;
```

- ▷ Define the context for the problem. In this case, we define parameters for the 2D and 3D experiment. In general, the parameters to define should be ν , dt , T , k and *#refinements*.

```
struct NavierContext{
    //Parameters for 2D experiment
    int max_order_steady = 6;
    int max_refinements_steady = 5;
    double kinvis_steady = 1.0 / 40.0;
    double t_final_steady = 50 * 0.001;
    double dt_steady = 0.001;
    double delta = 0.001;

    //Parameters for 3D experiment
    double kinvis_3d = 0.001;
    double t_final_3d = 8.0;
    double dt_3d = 1e-3;
} ctx;
```

- ▷ Define a velocity as a C function. This function represents the initial and boundary conditions for the velocity in the 3D experiment.

```
void vel_3d(const Vector &x, double t, Vector &u){
    double xi = x(0);
    double yi = x(1);
    double zi = x(2);
    double U = 2.25;
    if(xi <= 1e-8){
        u(0) = 16.0 * U * yi * zi * sin(M_PI * t / 8.0) *
            (0.41 - yi) * (0.41 - zi) / pow(0.41, 4.0);
    }else{ u(0) = 0.0; }
    u(1) = 0.0;
    u(2) = 0.0;}
```

- ▷ Define another velocity as a C function. This function represents the boundary condition, u_0 , for the 2D experiment.

```
void vel_steady(const Vector &x, double t, Vector &u){
    double reynolds = 1.0 / ctx.kinvis_steady;
    double lam = 0.5 * reynolds - sqrt(0.25 * reynolds *
        reynolds + 4.0 * M_PI * M_PI);
    double xi = x(0);
    double yi = x(1);
    u(0) = 1.0 - exp(lam * xi) * cos(2.0 * M_PI * yi);
    u(1) = lam / (2.0 * M_PI) * exp(lam * xi)
        * sin(2.0 * M_PI * yi);}
```

- ▷ Define a third velocity as a C function. This function represents the initial condition, u_i , for the 2D experiment.

```
void vel(const Vector &x, double t, Vector &u){
    double reynolds = 1.0 / ctx.kinvis_steady;
    double lam = 0.5 * reynolds - sqrt(0.25 * reynolds *
        reynolds + 4.0 * M_PI * M_PI);
    double xi = x(0);
    double yi = x(1);
    double delta = ctx.delta;
    u(0) = 1.0 - exp(lam * xi) * cos(2.0 * M_PI * yi);
    u(1) = lam / (2.0 * M_PI) * exp(lam * xi) * sin(2.0
        * M_PI * yi);
    u(0) = u(0)+delta*(xi+0.5)*(xi-1)*(yi+0.5)*(yi-1.5);
    u(1) = u(1)+delta*(xi+0.5)*(xi-1)*(yi+0.5)*(yi-1.5);
}
```

- ▷ Begin a function that receives an order and the amount of refinements. This function will compute the 2D experiment. First, it defines the mesh shown on figure 17.

```
void NS_steady(int order, int refinement){
    Mesh *mesh = new Mesh(2, 4, Element::QUADRILATERAL,
        false, 1.5, 2.0);
    mesh->EnsureNodes();
    GridFunction *nodes = mesh->GetNodes();
    *nodes == 0.5;
```

- ▷ Refine the mesh and create the parallel version of the mesh.

```
for (int i = 0; i < refinement; ++i){
    mesh->UniformRefinement();}
auto *pmesh = new ParMesh(MPI_COMM_WORLD, *mesh);
delete mesh;
```

- ▷ Create the NavierSolver object (receives the mesh, k , and ν as parameters). Set the initial condition, the function `vel`. And set the boundary condition, the function `vel_steady`.

```
//Create the flow solver
NavierSolver flowsolver(pmesh, order, ctx.
    kinvis_steady);
flowsolver.EnablePA(true);

//Set the initial condition
ParGridFunction *u_ic = flowsolver.
    GetCurrentVelocity();
VectorFunctionCoefficient u_excoeff(pmesh->
    Dimension(), vel);
u_ic->ProjectCoefficient(u_excoeff);

//Add Dirichlet boundary conditions to velocity
    space
Array<int> attr(pmesh->bdr_attributes.Max());
attr = 1;
flowsolver.AddVelDirichletBC(vel_steady, attr);
```

- ▷ Set up the problem (dt) and define the ParGridFunctions to store the solution.

```
double t = 0.0;
bool last_step = false;
flowsolver.Setup(ctx.dt_steady);
ParGridFunction *u_gf = flowsolver.
    GetCurrentVelocity();
ParGridFunction *p_gf = flowsolver.
    GetCurrentPressure();
```

- ▷ Create the ParaView file and associate the variables to save the solution.

```
ParaViewDataCollection pvdc("STEADY"+to_string(order
)+to_string(refinement), pmesh);
pvdc.SetDataFormat(VTKFormat::BINARY32);
pvdc.SetHighOrderOutput(true);
pvdc.SetLevelsOfDetail(order);
pvdc.SetCycle(0);
pvdc.SetTime(t);
pvdc.RegisterField("velocity", u_gf);
pvdc.RegisterField("pressure", p_gf);
pvdc.Save();
```

- ▷ Iterate from $t = 0$ to $t = T$. For each iteration, the step is taken and saved in the ParaView file.

```

for (int step = 0; !last_step; ++step){
    //Check for final step
    if (t + ctx.dt_steady >= ctx.t_final_steady -
        ctx.dt_steady / 2){last_step = true;}

    //Do the step
    flowsolver.Step(t, ctx.dt_steady, step);

    //Save paraview information
    pvdc.SetCycle(step);
    pvdc.SetTime(t);
    pvdc.Save();}
delete pmesh;}

```

- ▷ Begin a function that receives an order and the amount of refinements. This function will compute the 3D experiment. First, it defines the mesh associated to the domain shown on figure 21. This part of the code was planned to be used with different orders and refinement levels, however, it was only run with one case.

```

void NS_3D(int order, int refinement){
    Mesh *mesh = new Mesh("box-cylinder.mesh");
}

```

- ▷ Refine the mesh and create the parallel version of the mesh.

```

for (int i = 0; i < refinement; ++i){
    mesh->UniformRefinement();}
auto *pmesh = new ParMesh(MPI_COMM_WORLD, *mesh);
delete mesh;

```

- ▷ Create the NavierSolver object. Set the initial condition, the function `vel_3d`. And set the boundary condition, the function `vel_3d` too. Notice that the boundary condition is only applied to some parts of the mesh.

```
//Create the flow solver
NavierSolver flowsolver(pmesh, order, ctx.kinvis_3d)
;
flowsolver.EnablePA(true);

//Set the initial condition
ParGridFunction *u_ic = flowsolver.
    GetCurrentVelocity();
VectorFunctionCoefficient u_excoeff(pmesh->
    Dimension(), vel_3d);
u_ic->ProjectCoefficient(u_excoeff);

//Add Dirichlet boundary conditions to velocity
//space restricted to selected attributes on the
//mesh
Array<int> attr(pmesh->bdr_attributes.Max());
attr[0] = 1; //Inlet
attr[2] = 1; //Walls
flowsolver.AddVelDirichletBC(vel_3d, attr);
```

- ▷ Set up the problem (dt) and define the ParGridFunctions to store the solution.

```
double t = 0.0;
bool last_step = false;
flowsolver.Setup(ctx.dt_3d);
ParGridFunction *u_gf = flowsolver.
    GetCurrentVelocity();
ParGridFunction *p_gf = flowsolver.
    GetCurrentPressure();
```

- ▷ Create the ParaView file and associate the variables to save the solution.

```

ParaViewDataCollection pvdc("3D"+to_string(order)+
    to_string(refinement), pmesh);
pvdc.SetDataFormat(VTKFormat::BINARY32);
pvdc.SetHighOrderOutput(true);
pvdc.SetLevelsOfDetail(order);
pvdc.SetCycle(0);
pvdc.SetTime(t);
pvdc.RegisterField("velocity", u_gf);
pvdc.RegisterField("pressure", p_gf);
pvdc.Save();

```

- ▷ Iterate from $t = 0$ to $t = T$. For each iteration, the step is taken and saved in the ParaView file.

```

for(int step = 0; !last_step; ++step){
    //Check for final step
    if(t + ctx.dt_3d >= ctx.t_final_3d - ctx.dt_3d /
        2){last_step = true;}

    //Do the step
    flowsolver.Step(t, ctx.dt_3d, step);

    //Save paraview information every 10 steps
    if (step % 10 == 0){
        pvdc.SetCycle(step);
        pvdc.SetTime(t);
        pvdc.Save();}
delete pmesh;}

```

- ▷ Finally, define the main function with a MPI session (for parallel computation). The 2D experiment corresponds to the use of the function `NS_steady` while iterating through orders and refinement levels. And, the 3D experiment is running the function `NS_3D` with order 4. No refinements were done because the original mesh (done by MFEM [5]) gives a good solution already.

```
int main(int argc, char *argv[]) {

    //Define mpi sessions (parallel programming)
    MPI_Session mpi(argc, argv);

    //Steady experiment
    for(int order=1; order<=ctx.max_order_steady; order++)
    {
        for(int refinements=0; refinements<=ctx.
            max_refinements_steady; refinements++){
            NS_steady(order, refinements);
        }
    }

    //3D experiment
    NS_3D(4, 0);
}
```

Note: This code was run using the command `mpirun -n 4`, which uses 4 cores of the computer.